
```
HermesApplication.java
package com.hermes.android;
import com.hermes.android.toolprovider.ModelToolProvider;

import com.hermes.android.browser.BrowserToolProvider;

import com.hermes.android.toolprovider.OcrToolProvider;

import com.hermes.android.toolprovider.MemoryToolProvider;

import com.hermes.android.toolprovider.ChatMemoryProvider;

import com.hermes.android.toolprovider.FileOpsProvider;

import com.hermes.android.toolprovider.HermesGapProvider;

import com.hermes.android.toolprovider.UtilityToolProvider;

import com.hermes.android.toolprovider.ExtraToolProvider;

import com.hermes.android.toolprovider.HermesToolProvider;

import com.hermes.android.toolprovider.SystemToolProvider;

import com.hermes.android.scene.FastSceneProvider;

import com.hermes.android.workflow.Workflows;

import android.app.Application;

import com.hermes.android.agent.PromptLoader;
import com.hermes.android.agent.Journal;
import com.hermes.android.llama.SemanticReranker;
import com.hermes.android.ai.AiProviderConfig;
import com.hermes.android.model.ModelRegistry;
import com.hermes.android.model.ModelConfig;
import org.json.JSONArray;
import org.json.JSONObject;

public class HermesApplication extends Application {

    private static HermesApplication instance;
    public static HermesApplication getInstance() { return instance; }
    public static android.content.Context getAppContext() { return instance; }

    @Override
    public void onCreate() {
        super.onCreate();
        instance = this;
        /* W4: WebView 预热 - 预创建触发 Chromium 进程启动 (冷启动最大瓶颈, 首屏提速; 保留复用) */
```

```

try {
    android.webkit.WebView prewarm = new android.webkit.WebView(this);
    prewarm.loadDataWithBaseURL(null, "", "text/html", "UTF-8", null);
} catch (Exception ignored) {}
/* 工单 28 语义召回: 端侧 embedding 增强 Journal.search—装配重排器。
   失败静默 (reranker 保持 null → search 纯子串保底, 零回归)。 */
try {
    Journal.setSearchReranker(new SemanticReranker(this));
} catch (Exception ignored) {}
// 仅 debuggable 包开启 WebView 远程调试 (chrome://inspect / CDP), release 关闭
if ((getApplicationInfo().flags & android.content.pm.ApplicationInfo.FLAG_DEBUGGAB
    android.webkit.WebView.setWebContentsDebuggingEnabled(true);
// debug 宿主: ContextProvider 情境传感器层 v1 — 插拔充电器/WiFi 即落盘到 filesDir/context/
// 零订阅=零 receiver (电池卫生铁律), debug 宿主须订阅四族以激活落盘
com.hermes.android.context.ContextProvider cp = new com.hermes.android.context
    this,
    new com.hermes.android.context.SensorSourceImpl(this),
    new com.hermes.android.context.ContextPersister(
        new java.io.File(getFilesDir(), "context"))
);
cp.start();
// 激活四族 receiver (no-op 回调, 落盘由 ContextProvider 内部 fire() 路径完成)
com.hermes.android.context.TransitionListener noop = e -> {};
cp.subscribe(noop, com.hermes.android.context.ContextEventType.WIFI_CONNECTED,
    com.hermes.android.context.ContextEventType.WIFI_DISCONNECTED);
cp.subscribe(noop, com.hermes.android.context.ContextEventType.POWER_CONNECTED
    com.hermes.android.context.ContextEventType.POWER_DISCONNECTED);
cp.subscribe(noop, com.hermes.android.context.ContextEventType.SCREEN_ON,
    com.hermes.android.context.ContextEventType.SCREEN_OFF);
cp.addBatteryListener(noop, 80, true); // 升穿 80% (充电到 80%)
cp.addBatteryListener(noop, 20, false); // 降穿 20% (低电量告警)
}
TokenMeter.init(this);
MigrationManager.run(this);
RulesManager.init(this);
ErrorReporter.init(this);
PromptLoader.init(this);
com.hermes.android.mcp.McpProvider.init(this);
ModelToolProvider.init(this);
SystemToolProvider.init(this);
HermesToolProvider.init(this);
ExtraToolProvider.init(this);
UtilityToolProvider.init(this);
HermesGapProvider.init(this);
FileOpsProvider.init(this);
ChatMemoryProvider.init(this);
FastSceneProvider.init(this);
MemoryToolProvider.init(this);
com.hermes.android.toolprovider.CausalToolProvider.init(this); // 因果记忆编译器 (四元组+哈希
OcrToolProvider.init(this); // OCR 集成面 (模型/llama-server 归 P4)

```

```

com.hermes.android.toolprovider.PbToolProvider.init(this); // MCP 工厂首品种: PocketBa
com.hermes.android.toolprovider.PartnerToolProvider.init(this); // 微信支付服务商进件 wxp
Workflows.init(this); // 触发器引擎 (workflow 纯 JVM + ConnectWeb 集成壳)
/* 阶段3 A0: MOV 入站监听 (127.0.0.1:8388, daemon→MOV 回调地基)。 */
    幂等启动; 端口占用/进程竞争静默降级 (仅主进程持监听)。 */
com.hermes.android.serve.MovInboundServer.ensureRunning(this);
/* 阶段3 A3: MOV 工具清单推送 daemon (/exchange/mov-tools.json, serve GET /tools 读) */
com.hermes.android.serve.ToolManifest.writeToExchange(this);
/* MOV 能力 MCP 化 P1: 进程级 MCP server (127.0.0.1:8389/mcp, 官方 SDK, 只读工具集)。 */
    幂等启动; mapper 缺失/端口占用静默降级 (不阻塞 onCreate)。 */
com.hermes.android.mcp.server.MovMcpServer.ensureRunning(this);
/* HtmlViewer 多文件 HTML 项目支持: 房间静态文件服务器 (127.0.0.1:8787, file:// 无法执行
    file:// JS 子资源, 多文件项目走 http 加载)。幂等启动; 端口占用静默降级。 */
com.hermes.android.serve.RoomStaticServer.ensureRunning(this);
/* APK 预置 rootfs: 首启/每次启动后台自动就绪 (isReady 幂等, 无 UI 无感) —
    rootfs 未就绪 → assets 种子自动解压; READY 后联动装 Hermes (装进 rootfs, 需网络, 失败下次补)。 */
ensureLinuxBootstrap();
ensureDefaultsAndCleanup();
}

/** 后台静默引导 Linux + Hermes (用户无感; rootfs 已就绪则跳过, 坏了自己修)。 */
private void ensureLinuxBootstrap() {
    final android.content.Context ctx = this;
    new Thread(() -> {
        try {
            final com.hermes.android.linux.RootfsManager rm = new com.hermes.android.l
            rm.setListener((s, p, m) -> {
                // rootfs READY 后 ensure Hermes: 种子含 venv 走快路径 (~3s), 无 venv 回落完整装
                if (s == com.hermes.android.linux.RootfsManager.State.READY) {
                    new com.hermes.android.linux.HermesInstaller(ctx).ensureReady();
                }
            });
            if (!rm.isReady()) {
                rm.install();
            } else {
                // rootfs 已就绪 → 直接 ensure Hermes (venv 在 → 快路径; 缺 → 完整装)
                new com.hermes.android.linux.HermesInstaller(ctx).ensureReady();
            }
        } catch (Exception e) {
            android.util.Log.w("MOV", "Linux/Hermes 自动就绪失败: " + e.getMessage());
        }
    }, "linux-bootstrap").start();
}

/**
 * 清理历史遗留的已弃用模型名。
 * getProvider() 默认返回 "deepseek", 无需显式写入。
 * 不再预置 API Key—用户需在 AI 设置中自行填写。
 */
private void ensureDefaultsAndCleanup() {

```

```
AiProviderConfig config = new AiProviderConfig(this);
// 清理历史遗留的已弃用模型名 (deepseek-chat / deepseek-reasoner 已于 7/24 弃用)
String m = config.getModel();
if ("deepseek-chat".equals(m) || "deepseek-reasoner".equals(m)) {
    config.setModel("");
}

// MCP: 模型配置持久化 (重装不丢模型)
autoImportModels();
// 启动时同步模型到内嵌 Hermes
syncToHermes();
}

/** 启动时自动从内部存储导入模型配置 (MCP 口), 重装不丢 (-r 升级保留内部数据)。
    技术债修复 (工单 5): 原落外部存储 getExternalFilesDir 明文含 apiKey, 改内部 filesDir;
    顺手清掉历史遗留的外部明文文件。 */
private void autoImportModels() {
    try {
        java.io.File legacy = new java.io.File(getExternalFilesDir(null), "mcp_models.");
        if (legacy.exists()) legacy.delete(); // 旧外部明文文件清理
        java.io.File mcpFile = new java.io.File(getFilesDir(), "mcp_models.json");
        if (!mcpFile.exists()) return;
        String json = new String(java.nio.file.Files.readAllBytes(mcpFile.toPath()), "
        JSONArray arr = new JSONArray(json);
        ModelRegistry registry = ModelRegistry.getInstance(this);
        int imported = 0;
        for (int i = 0; i < arr.length(); i++) {
            JSONObject o = arr.getJSONObject(i);
            boolean exists = false;
            for (ModelConfig c : registry.list()) {
                if (o.optString("provider", "").equals(c.provider)
                    && o.optString("model", "").equals(c.model)) { exists = true; break
            }
            if (!exists) {
                ModelConfig mc = new ModelConfig();
                mc.provider = o.optString("provider", "");
                mc.name = o.optString("name", mc.provider);
                mc.model = o.optString("model", "");
                mc.apiKey = o.optString("apiKey", "");
                mc.baseUrl = o.optString("baseUrl", "");
                mc.enabled = true;
                mc.isDefault = o.optBoolean("isDefault", false);
                mc.role = o.optString("role", "通用");
                registry.add(mc);
                imported++;
            }
        }
        if (imported > 0) {
            android.util.Log.i("MOV", "MCP: 从外部存储导入 " + imported + " 个模型");
        }
    }
}
```

```

    } catch (Exception e) {
        android.util.Log.w("MOV", "MCP 导入失败: " + e.getMessage());
    }
}

/** 启动时将默认模型同步到内嵌 Hermes (~/.hermes/.env + config.yaml) */
private void syncToHermes() {
    new Thread(() -> {
        try {
            if (com.hermes.android.linux.HermesInstaller.isReady(this)) {
                com.hermes.android.linux.HermesInstaller.writeModelConfig(this);
            }
        } catch (Exception ignored) { /* 非致命 */ }
    }, "hermes-sync").start();
}

/** MCP 口: JS 调此方法导出当前模型配置到外部存储 */
@android.webkit.JavascriptInterface
public String exportModels() {
    try {
        ModelRegistry registry = ModelRegistry.getInstance(this);
        JSONArray arr = new JSONArray();
        for (ModelConfig c : registry.list()) {
            JSONObject o = new JSONObject();
            o.put("provider", c.provider);
            o.put("name", c.name);
            o.put("model", c.model);
            o.put("apiKey", c.apiKey);
            o.put("baseUrl", c.baseUrl != null ? c.baseUrl : "");
            o.put("isDefault", c.isDefault);
            o.put("role", c.role != null ? c.role : "通用");
            arr.put(o);
        }
        /* 技术债修复 (工单 5): 写内部 filesDir (getFilesDir), 不再落外部存储明文含 key。 */
        java.io.File mcpFile = new java.io.File(getFilesDir(), "mcp_models.json");
        java.nio.file.Files.write(mcpFile.toPath(), arr.toString(2).getBytes("UTF-8"))
        return "{\"ok\":true,\"count\":" + arr.length() + ", \"path\":\"" + mcpFile.get
    } catch (Exception e) {
        return "{\"ok\":false,\"error\":\"" + e.getMessage() + "\"}";
    }
}
}

HermesActivity.java
package com.hermes.android;

import android.Manifest;
import android.annotation.SuppressLint;
import android.content.Context;
import android.content.Intent;
import android.content.IntentFilter;

```

```
import android.content.pm.PackageManager;
import android.net.Uri;
import android.net.wifi.WifiInfo;
import android.net.wifi.WifiManager;
import android.os.BatteryManager;
import android.os.Build;
import android.os.Bundle;
import android.os.Handler;
import android.os.Looper;
import android.provider.Settings;
import android.util.Log;
import android.webkit.JavascriptInterface;
import android.webkit.WebChromeClient;
import android.webkit.WebSettings;
import android.webkit.WebView;
import android.webkit.WebViewClient;
import android.widget.Toast;

import androidx.activity.OnBackPressedCallback;
import androidx.activity.result.ActivityResultLauncher;
import androidx.activity.result.contract.ActivityResultContracts;
import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.ActivityCompat;
import androidx.core.content.ContextCompat;

import com.hermes.android.ai.AiClient;
import com.hermes.android.ai.AiProviderConfig;
import com.hermes.android.agent.HealthMonitor;
import com.hermes.android.agent.HealthProbeFactory;
import com.hermes.android.skill.SkillStore;
import com.hermes.android.vault.KeyguardGate;
import com.hermes.android.capability.CapabilityExecutor;
import com.hermes.android.security.PaymentGate;
import com.hermes.android.vault.SecureVault;

import org.json.JSONArray;
import org.json.JSONObject;

import java.io.File;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

/**
 * Hermes v2.0 – 客户端壳 Activity。
 *
 * P0-1: AI 调用异步化 (后台线程 + evaluateJavascript 回调)
 * P0-3: TTS 生命周期管理 (onDestroy shutdown)
```

```
* P2-13: chatHistory 持久化
* P2-14: getDeviceInfo 缓存
* P2-17: OnBackPressedCallback 替代 deprecated onBackPressed
*/
public class HermesActivity extends AppCompatActivity {

    private static final String TAG = "MOV";
    private static final int PERM_REQUEST = 1001;
    private static final String HISTORY_KEY = "chat_history_json";
    /** P0: 文件选择器拷贝上限 (与 BridgeValidator SMB 一致) */
    private static final long MAX_COPY_BYTES = 5L * 1024 * 1024;

    private WebView shell;
    /** P5 双副本保险柜: 加密存储 + 解锁闸 (复用 PaymentGate 锁屏确认) */
    private SecureVault secureVault;
    private PaymentGate paymentGate;
    private KeyguardGate keyguardGate;
    /** H2: 工具/源周期探测器 (30min 轻探测 + 失败退避) */
    private HealthMonitor healthMonitor;
    /** P5: 当前活动实例 (vault 工具 handler 需访问本机 Keyguard 闸/保险柜; ConnectWebActivity.getCurrent 同款模
    private static volatile HermesActivity current;
    private final Handler uiHandler = new Handler(Looper.getMainLooper());
    private final CapabilityExecutor capabilityExecutor = new CapabilityExecutor();
    private AiProviderConfig aiConfig;
    private SkillStore skillStore;
    private com.hermes.android.model.ModelRegistry modelRegistry;
    private StorageManager storageManager;
    private com.hermes.android.mcp.McpServer mcpServer;
    private final List<AiClient.Message> chatHistory = new ArrayList<>();
    /** 工单 26: 当前 BridgeFactory (市场引擎生命周期钩子 onResume/onPause 用) */
    private com.hermes.android.bridge.BridgeFactory currentBridgeFactory;

    /** P0-1: 后台线程池 (AI / Council 异步调用) */
    private final ExecutorService aiExecutor = Executors.newFixedThreadPool(2);

    /** JS 侧当前是否打开某个房间 */
    private volatile boolean roomOpen = false;

    /** P1-8: 文件选择器回调 ID */
    private volatile String pendingFileCallbackId;
    /** Fix 1: 文件选择器目标房间 ID */
    private volatile String pendingFileRoomId;

    /** P1-8: 文件选择器 */
    private ActivityResultLauncher<Intent> filePickerLauncher;

    @SuppressWarnings("SetJavaScriptEnabled")
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
```

```
setContentView(R.layout.activity_hermes);

/* 状态栏 + 导航栏融入暗色主题 */
getWindow().setStatusBarColor(0xFF0f172a);
getWindow().setNavigationBarColor(0xFF0f172a);

aiConfig = new AiProviderConfig(this);
modelRegistry = com.hermes.android.model.ModelRegistry.getInstance(this);
storageManager = new StorageManager(this);
capabilityExecutor.init(this);
// P5: 双副本保险柜 - SecureVault 加密存储 + Keyguard 解锁闸 (复用 PaymentGate 锁屏确认)
secureVault = new SecureVault(this);
paymentGate = new PaymentGate(this);
keyguardGate = new KeyguardGate();
// H2: 工具/源周期探测器 (30min 轻探测 + 失败退避; 后台线程零打扰)
healthMonitor = new HealthMonitor(HealthProbeFactory.build(this));
healthMonitor.start();
registerNetWatcher(); // 流式断线感知 (断网秒级中止+提示)
skillStore = new SkillStore(this);
skillStore.ensureSeeded(this); // 批2: 释放内置技能到 filesDir/skills (skip-if-exists)

// 全局未捕获异常 -> 交还默认处理器
final Thread.UncaughtExceptionHandler defaultHandler =
    Thread.getDefaultUncaughtExceptionHandler();
Thread.setDefaultUncaughtExceptionHandler((thread, ex) -> {
    if (defaultHandler != null) defaultHandler.uncaughtException(thread, ex);
});

// Prompt 文件加载器注入 context
com.hermes.android.agent.AgentLoop.setAppContext(this);

// P2-13: 恢复聊天历史
restoreChatHistory();

// P1-8: 注册文件选择器
filePickerLauncher = registerForActivityResult(
    new ActivityResultContracts.StartActivityForResult(), result -> {
        if (pendingFileCallbackId == null) return;
        String cbId = pendingFileCallbackId;
        String roomId = pendingFileRoomId;
        pendingFileCallbackId = null;
        pendingFileRoomId = null;
        if (result.getResultCode() == RESULT_OK && result.getData() != null) {
            Uri uri = result.getData().getData();
            if (uri != null) {
                // Fix 1: 复制到房间目录; feat/wxpay-applyment: 等复制完成再把房间路径带给 JS
                if (roomId != null && !roomId.isEmpty()) {
                    String info = getFileInfoJson(uri);
                    copyFileToRoom(roomId, uri, roomPath -> {
                        try {
```

```

        JSONObject j = new JSONObject(info == null ? "{}"
        if (roomPath != null) {
            j.put("roomPath", roomPath);
        } else {
            j.put("ok", false).put("error", "复制到房间目录失败 (文件名)");
        }
        evalJs("window._hermesCb('" + cbId + "'," + j + ")");
    } catch (Exception ex) {
        evalJs("window._hermesCb('" + cbId
            + "',{"ok\":false,\"error\":\"\复制结果回传失败\"}");
    }
    });
} else {
    String info = getFileInfoJson(uri);
    evalJs("window._hermesCb('" + cbId + "'," + info + ")");
}
return;
}
}
evalJs("window._hermesCb('" + cbId + "',null)");
});

current = this;
shell = findViewById(R.id.shell);
com.hermes.android.ConnectWebActivity.setShellView(shell);
WebSettings s = shell.getSettings();
s.setJavaScriptEnabled(true);
s.setMediaPlaybackRequiresUserGesture(false); // 2C: B 站内嵌播放器 autoplay
s.setDomStorageEnabled(true);
s.setLoadWithOverviewMode(true);
s.setUseWideViewPort(true);
s.setTextZoom(100);
// P0: 关闭文件/content 访问, 防 WebView 越权读本地文件
// (file:///android_asset 加载在 allowFileAccess=false 下仍可用)
s.setAllowFileAccess(false);
s.setAllowContentAccess(false);
// 防升级安装后 WebView 复用旧版 JS/CSS 缓存导致前后端文件版本错位 (assets 本地加载, 禁缓存无性能损失)
s.setCacheMode(WebSettings.LOAD_NO_CACHE);

shell.setBackgroundColor(0xFF6F6F7);
// P0: URL 白名单, 仅放行本地 assets
shell.setWebViewClient(new WebViewClient() {
    @Override
    public boolean shouldOverrideUrlLoading(WebView view, String url) {
        return url == null || !url.startsWith("file:///android_asset/");
    }
}
/* W4: 首屏埋点 - 进程启动到 onPageFinished ms (真机 logcat MOV-DIAG, 达标 <1500ms) */
@Override
public void onPageFinished(WebView view, String url) {
    super.onPageFinished(view, url);

```

```

        long start = android.os.Process.getStartElapsedRealtime();
        long ms = android.os.SystemClock.elapsedRealtime() - start;
        android.util.Log.i("MOV-DIAG", "first-frame " + ms + "ms");
    }
});
shell.setWebChromeClient(new WebChromeClient() {
    /* Fix: WebView 默认不处理 confirm()/prompt() - confirm 恒返回 false、prompt 恒返回 null,
       导致模板使用命名等流程静默失败。用原生对话框承接。 */
    @Override
    public boolean onJsConfirm(WebView view, String url, String message,
        android.webkit.JsResult result) {
        new androidx.appcompat.app.AlertDialog.Builder(HermesActivity.this)
            .setMessage(message)
            .setPositiveButton(android.R.string.ok, (d, w) -> result.confirm())
            .setNegativeButton(android.R.string.cancel, (d, w) -> result.cance
            .setOnCancelListener(d -> result.cancel())
            .show();
        return true;
    }

    @Override
    public boolean onJsPrompt(WebView view, String url, String message, String def
        android.webkit.JsPromptResult result) {
        final android.widget.EditText input = new android.widget.EditText(HermesAc
        input.setText(defaultValue);
        int pad = (int) (16 * getResources().getDisplayMetrics().density);
        input.setPadding(pad, 0, pad, 0);
        new androidx.appcompat.app.AlertDialog.Builder(HermesActivity.this)
            .setMessage(message)
            .setView(input)
            .setPositiveButton(android.R.string.ok,
                (d, w) -> result.confirm(input.getText().toString()))
            .setNegativeButton(android.R.string.cancel, (d, w) -> result.cance
            .setOnCancelListener(d -> result.cancel())
            .show();
        return true;
    }
});
com.hermes.android.bridge.BridgeFactory bf = new com.hermes.android.bridge.BridgeF
currentBridgeFactory = bf;
shell.addJavascriptInterface(bf, "HermesBridge");

// 启动 MCP 服务 (旧 McpServer 工具表 = @McpExpose 标注的只读安全方法 + mov.models 脱敏只读;
//   R8 后不再自动暴露全部 @JavascriptInterface, 新桥方法须显式 @McpExpose 才上网)
mcpServer = new com.hermes.android.mcp.McpServer();
mcpServer.start(bf);
shell.loadUrl("file:///android_asset/hermes-shell.html");

requestPermissions();

```

```

// P2-17: OnBackPressedCallback
getOnBackPressedDispatcher().addCallback(this, new OnBackPressedCallback(true) {
    @Override
    public void handleOnBackPressed() {
        if (roomOpen) {
            evalJs("if(window.curRoomId!=null){genCounter++;curRoomId=null;" +
                "if(window.HermesBridge)HermesBridge.setRoomOpen('');" +
                "setTab('chat');showView('view-rooms');renderRooms();}");
            roomOpen = false;
        } else {
            setEnabled(false);
            getOnBackPressedDispatcher().onBackPressed();
        }
    }
});
}

@Override
protected void onResume() {
    super.onResume();
    shell.post(() -> evalJs(
        "if(window.refreshRuntime)refreshRuntime();"));
    /* 工单 26: 市场引擎（卖家查单轮询 + 双身份 A2A 轮询）前台启动 */
    try {
        com.hermes.android.bridge.BridgeFactory bf = currentBridgeFactory;
        if (bf != null && bf.getMarket() != null) bf.getMarket().engine().onResume();
    } catch (Exception ignored) {}
}

@Override
protected void onPause() {
    /* 工单 26: 退后台停市场轮询（查单/双身份 A2A） */
    try {
        com.hermes.android.bridge.BridgeFactory bf = currentBridgeFactory;
        if (bf != null && bf.getMarket() != null) bf.getMarket().engine().onPause();
    } catch (Exception ignored) {}
    super.onPause();
}

/** P5: 保险柜 Keyguard 结果转发（REQUEST_CODE_PAYMENT_CONFIRM → paymentGate.handleActivityResult）
@Override
protected void onActivityResult(int requestCode, int resultCode, Intent data) {
    if (paymentGate != null && paymentGate.handleActivityResult(requestCode, resultCode, data)) return;
    /* W4: 语音识别结果（9002, 与 PaymentGate 9001 错开）→ 固定回调 _movStt（JS bindMic 监听） */
    if (requestCode == 9002) {
        if (resultCode == RESULT_OK && data != null) {
            java.util.ArrayList<String> results = data.getStringArrayListExtra(
                android.speech.RecognizerIntent.EXTRA_RESULTS);
            String text = (results != null && !results.isEmpty()) ? results.get(0) : "";
            evalJsPublic("window._movStt({ok:" + (!text.isEmpty()) + ",text:" + org.js

```

```

        return;
    }
    evalJsPublic("window._movStt({ok:false,error:'speech_cancelled'})");
    return;
}
super.onActivityResult(requestCode, resultCode, data);
}

@Override
protected void onDestroy() {
    // P0-3: 释放 TTS
    capabilityExecutor.shutdown();
    // H2: 停周期探测器（后台线程零打扰）
    if (healthMonitor != null) healthMonitor.stop();
    // 流式断线感知：注销网络监听
    unregisterNetWatcher();
    // P0-1: 关闭线程池
    aiExecutor.shutdownNow();
    // P2-13: 保存聊天历史
    saveChatHistory();
    if (mcpServer != null) { mcpServer.stop(); }
    if (current == this) current = null;
    if (shell != null) {
        // P0: destroy 前先从父布局移除并移除 JS 桥，防泄漏与悬空引用
        shell.removeJavascriptInterface("HermesBridge");
        if (shell.getParent() instanceof android.view.ViewGroup) {
            ((android.view.ViewGroup) shell.getParent()).removeView(shell);
        }
        shell.destroy();
        shell = null;
    }
    super.onDestroy();
}

// ===== 流式断线感知 (CONTRACT_STREAMING 2.5) =====

private android.net.ConnectivityManager.NetworkCallback netCallback;

/** 断网秒级中止在途请求 + 提示；恢复提示可续跑 */
private void registerNetWatcher() {
    try {
        android.net.ConnectivityManager cm =
            (android.net.ConnectivityManager) getSystemService(CONNECTIVITY_SERVICE);
        if (cm == null) return;
        netCallback = new android.net.ConnectivityManager.NetworkCallback() {
            @Override
            public void onLost(android.net.Network network) {
                com.hermes.android.ai.AiClient.cancelAllActive();
                evalJs("window._movNet&&window._movNet(false)");
            }
        };
    }
}

```

```

        @Override
        public void onAvailable(android.net.Network network) {
            evalJs("window._movNet&&window._movNet(true)");
        }
    };
    cm.registerDefaultNetworkCallback(netCallback);
} catch (Exception e) {
    android.util.Log.w("HermesActivity", "registerNetWatcher: " + e.getMessage());
}
}

private void unregisterNetWatcher() {
    if (netCallback == null) return;
    try {
        android.net.ConnectivityManager cm =
            (android.net.ConnectivityManager) getSystemService(CONNECTIVITY_SERVICE);
        if (cm != null) cm.unregisterNetworkCallback(netCallback);
    } catch (Exception ignored) {}
    netCallback = null;
}

/** 在主线程安全执行 JS (onDestroy 后自动跳过, 避免异步回调 NPE) */
private void evalJs(String script) {
    uiHandler.post(() -> {
        if (shell != null && !isDestroyed() && !isFinishing()) {
            shell.evaluateJavascript(script, null);
        }
    });
}

/** Bridge 子模块用的公开入口 */
public void evalJsPublic(String script) {
    evalJs(script);
}

/** Bridge 子模块用的 URL 加载入口 */
public void loadUrlPublic(String url) {
    runOnUiThread(() -> shell.loadUrl(url));
}

// ===== P2-13: 聊天历史持久化 =====

private void saveChatHistory() {
    try {
        JSONArray arr = new JSONArray();
        synchronized (chatHistory) {
            for (AiClient.Message m : chatHistory) {
                arr.put(new JSONObject().put("role", m.role).put("content", m.content));
            }
        }
    }
}

```

```

        getPreferences(MODE_PRIVATE).edit()
            .putString(HISTORY_KEY, arr.toString()).apply();
    } catch (Exception e) {
        Log.w(TAG, "saveChatHistory: " + e.getMessage());
    }
}

private void restoreChatHistory() {
    try {
        String json = getPreferences(MODE_PRIVATE).getString(HISTORY_KEY, null);
        if (json == null) return;
        JSONArray arr = new JSONArray(json);
        synchronized (chatHistory) {
            chatHistory.clear();
            for (int i = 0; i < arr.length(); i++) {
                JSONObject o = arr.getJSONObject(i);
                chatHistory.add(new AiClient.Message(
                    o.getString("role"), o.getString("content")));
            }
        }
    } catch (Exception e) {
        Log.w(TAG, "restoreChatHistory: " + e.getMessage());
    }
}

// ===== P1-8: 文件信息 =====

/** Fix 1: 从 URI 复制文件到房间目录 (P0: 文件名消毒 + roomId 校验 + 子线程 + 5MB 上限) */
private void copyFileToRoom(String roomId, Uri uri) {
    copyFileToRoom(roomId, uri, null);
}

/**
 * 复制 URI 到房间目录 (消毒 + 5MB 上限 + 路径越界校验)。
 * done != null 时: 复制完成后回调房间文件绝对路径; 任何失败回调 null (feat/wxpay-applyment 图片上传需要真实路
径)。
 */
private void copyFileToRoom(String roomId, Uri uri, java.util.function.Consumer<String>
    // P0: roomId 与 StorageManager/BridgeValidator 同规则
    if (com.hermes.android.bridge.BridgeValidator.checkRoomId(roomId) != null) {
        Log.w(TAG, "copyFileToRoom: 非法房间 ID " + roomId);
        if (done != null) done.accept(null);
        return;
    }
    String name = "unknown";
    try (android.database.Cursor c = getContentResolver().query(
        uri, null, null, null, null)) {
        if (c != null && c.moveToFirst()) {
            int idx = c.getColumnIndex(android.provider.OpenableColumns.DISPLAY_NAME);
            if (idx >= 0) name = c.getString(idx);
        }
    }
}

```

```

    }
    // P0: DISPLAY_NAME 消毒 - 拒绝路径分隔与 "..", 只保留文件名本体
    if (name == null || name.contains("/") || name.contains("\\"))
        || name.contains("..") || name.isEmpty() {
        Log.w(TAG, "copyFileToRoom: 非法文件名 " + name);
        if (done != null) done.accept(null);
        return;
    }
    final String safeName = name;
    aiExecutor.execute() -> {
        try {
            java.io.File base = new java.io.File(storageManager.getRoomsDir(), roomId)
            base.mkdirs();
            java.io.File target = new java.io.File(base, safeName).getCanonicalFile();
            String basePath = base.getCanonicalFile().getPath();
            if (!target.getPath().startsWith(basePath + java.io.File.separator)) {
                Log.w(TAG, "copyFileToRoom: 路径越界 " + safeName);
                if (done != null) done.accept(null);
                return;
            }
        }
        long total = 0;
        try (java.io.InputStream is = getContentResolver().openInputStream(uri);
            java.io.FileOutputStream os = new java.io.FileOutputStream(target)) {
            byte[] buf = new byte[8192];
            int n;
            while ((n = is.read(buf)) != -1) {
                total += n;
                // P0: 与 BridgeValidator 5MB 上限一致, 超限拒绝
                if (total > MAX_COPY_BYTES) {
                    Log.w(TAG, "copyFileToRoom: 文件过大 (>5MB) " + safeName);
                    target.delete();
                    if (done != null) done.accept(null);
                    return;
                }
                os.write(buf, 0, n);
            }
            Log.i(TAG, "copyFileToRoom: " + safeName + " → " + target.getAbsolutePath());
            if (done != null) done.accept(target.getAbsolutePath());
        } catch (Exception e) {
            Log.w(TAG, "copyFileToRoom: " + e.getMessage());
            if (done != null) done.accept(null);
        }
    });
}

private String getFileInfoJson(Uri uri) {
    try {
        String name = "unknown";
        long size = 0;
    }

```

```

        String mime = getContentResolver().getType(uri);
        try (android.database.Cursor c = getContentResolver().query(
            uri, null, null, null, null)) {
            if (c != null && c.moveToFirst()) {
                int nameIdx = c.getColumnIndex(android.provider.OpenableColumns.DISPLAY_NAME);
                int sizeIdx = c.getColumnIndex(android.provider.OpenableColumns.SIZE);
                if (nameIdx >= 0) name = c.getString(nameIdx);
                if (sizeIdx >= 0) size = c.getLong(sizeIdx);
            }
        }
        return new JSONObject()
            .put("name", name)
            .put("size", size)
            .put("mime", mime != null ? mime : "application/octet-stream")
            .put("uri", uri.toString())
            .toString();
    } catch (Exception e) {
        return "null";
    }
}

// ===== 权限 =====

private void requestPermissions() {
    List<String> needed = new ArrayList<>();
    String[] perms = {
        Manifest.permission.CAMERA,
        Manifest.permission.ACCESS_FINE_LOCATION,
        Manifest.permission.READ_CONTACTS,
        Manifest.permission.READ_SMS,
        Manifest.permission.CALL_PHONE,
    };
    for (String p : perms) {
        if (ContextCompat.checkSelfPermission(this, p) != PackageManager.PERMISSION_GRANTED)
            needed.add(p);
    }
    if (Build.VERSION.SDK_INT >= 33) {
        if (ContextCompat.checkSelfPermission(this, Manifest.permission.POST_NOTIFICATIONS)
            != PackageManager.PERMISSION_GRANTED) {
            needed.add(Manifest.permission.POST_NOTIFICATIONS);
        }
    }
    if (!needed.isEmpty()) {
        ActivityCompat.requestPermissions(this, needed.toArray(new String[0]), PERM_REQUEST_NOTIFICATIONS)
    }
}

@Override
public void onRequestPermissionsResult(int requestCode, @NonNull String[] permissions,

```

```

        @NonNull int[] grantResults) {
    super.onRequestPermissionsResult(requestCode, permissions, grantResults);
    if (requestCode == PERM_REQUEST) {
        int granted = 0;
        for (int r : grantResults) if (r == PackageManager.PERMISSION_GRANTED) granted
        Log.i(TAG, "permissions granted: " + granted + "/" + grantResults.length);
    }
}

/* =====
 * Bridge 子模块 getter (供 BridgeFactory 使用)
 * ===== */
public CapabilityExecutor getCapabilityExecutor() { return capabilityExecutor; }
public AiProviderConfig getAiConfig() { return aiConfig; }
public com.hermes.android.model.ModelRegistry getModelRegistry() { return modelRegistr
public ExecutorService getAiExecutor() { return aiExecutor; }
public List<AiClient.Message> getChatHistory() { return chatHistory; }
public StorageManager getStorageManager() { return storageManager; }
/** P5: 当前活动实例 (vault 工具 handler 用; 无活动窗口返回 null → 工具报诚实卡) */
public static HermesActivity getCurrent() { return current; }
/** P5: 保险柜 (加密存储) 访问器 */
public SecureVault getSecureVault() { return secureVault; }
/** H2: 周期探测器 (设置页能力区块健康态数据源; 可手动重测) */
public HealthMonitor getHealthMonitor() { return healthMonitor; }
/** P5: Keyguard 解锁闸 + 复用 PaymentGate 的 Prompter (vault_unlock 工具/桥用) */
public KeyguardGate getKeyguardGate() { return keyguardGate; }
public KeyguardGate.Prompter getKeyguardPrompter() {
    return new KeyguardGate.PaymentGatePrompter(this, paymentGate);
}
public SkillStore getSkillStore() { return skillStore; }
public void saveChatHistoryPublic() { saveChatHistory(); }
public void setRoomOpenPublic(boolean open) { roomOpen = open; }
public void pickFilePublic(String cbId, String roomId) {
    runOnUiThread(() -> {
        // Fix: 单槽位 - 已有未完成的选择时直接回错误回调, 不覆盖前一个回调
        if (pendingFileCallbackId != null) {
            try {
                String errJson = new JSONObject()
                    .put("ok", false).put("error", "已有文件选择操作进行中").toString();
                evalJs("window._hermesCb('" + cbId + "', " + errJson + ")");
            } catch (Exception ex) {
                evalJs("window._hermesCb('" + cbId
                    + "', {"ok":false,\"error\":\"已有文件选择操作进行中\"})");
            }
        }
        return;
    })
    try {
        Intent intent = new Intent(Intent.ACTION_OPEN_DOCUMENT);
        intent.addCategory(Intent.CATEGORY_OPENABLE);
        intent.setType("*/*");
    }
}

```

```

        pendingFileCallbackId = cbId;
        pendingFileRoomId = roomId;
        filePickerLauncher.launch(intent);
    } catch (Exception e) {
        try {
            /* JSONObject 构造错误 JSON, 防 e.getMessage() 含引号注入 JS 字符串 */
            String errJson = new JSONObject()
                .put("ok", false).put("error", e.getMessage()).toString();
            evalJs("window._hermesCb('" + cbId + "'," + errJson + ")");
        } catch (Exception ex) {
            evalJs("window._hermesCb('" + cbId
                + "',{"ok\":false,\"error\":\'文件选择失败\'})");
        }
    }
});
}

}

BaseBridge.java
package com.hermes.android.bridge;

import com.hermes.android.HermesActivity;

/**
 * Bridge 基类 - 持有 Activity 引用, 子类通过它访问共享资源。
 */
public abstract class BaseBridge {
    protected final HermesActivity activity;

    public BaseBridge(HermesActivity activity) {
        this.activity = activity;
    }

    /** 在主线程安全执行 JS 回调 */
    protected void evalJs(String script) {
        activity.evalJsPublic(script);
    }
}

BridgeA2a.java
package com.hermes.android.bridge;

import android.webkit.JavascriptInterface;

import com.hermes.android.HermesActivity;
import com.hermes.android.a2a.A2aClient;

import org.json.JSONArray;
import org.json.JSONObject;

import java.io.File;

```

```

import java.io.FileWriter;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.util.List;

/**
 * A2A Bridge - 暴露 A2aClient 给 JS 商家页面。
 * 全部消息落 journal (audit:true, type 前缀 a2a_)。
 */
public class BridgeA2a extends BaseBridge {
    private final A2aClient client;
    private final File dataDir; // filesDir/a2a/

    public BridgeA2a(HermesActivity activity) {
        super(activity);
        this.client = new A2aClient(activity);
        this.dataDir = new File(activity.getFilesDir(), "a2a");
        this.dataDir.mkdirs();
    }

    @JavascriptInterface public String getDeviceId() { return client.getDeviceId(); }
    @JavascriptInterface public boolean isRegistered() { return client.isRegistered(); }

    /* — Relay 服务器地址（设置页 A2A 区块编辑） — */
    @JavascriptInterface public String getRelayUrl() { return client.baseUrl(); }
    @JavascriptInterface public void setRelayUrl(String url) { client.setBaseUrl(url); }

    @JavascriptInterface
    public void a2aOpenMerchant() {
        activity.loadUrlPublic("file:///android_asset/a2a/merchant.html");
    }

    @JavascriptInterface
    public void a2aRegister(String deviceId, String name, String cbId) {
        a2aRegister(deviceId, name, null, null, cbId); // 缺省兼容: role=buyer
    }

    /** 职业注册表注册: role (buyer|pro) + professions (工种数组 JSON), 缺省兼容 */
    @JavascriptInterface
    public void a2aRegister(String deviceId, String name, String role, String professionsJ
        org.json.JSONArray professions = null;
        if (professionsJson != null && !professionsJson.isEmpty()) {
            try { professions = new org.json.JSONArray(professionsJson); } catch (Exceptio
        }
        client.register(deviceId, name, role, professions, new A2aClient.Callback<String>()
            @Override public void onSuccess(String token) {
                evalJs("window._hermesCb('" + cbId + "',{"ok\":true,\"token\":\"" + token
            }
            @Override public void onFailure(java.io.IOException e) {
                evalJs("window._hermesCb('" + cbId + "',{"ok\":false,\"error\":\"" + esc(

```

```

    }
    });
}

/** 附近职业人发现 (relay query_pro) : 按 craft + radiusKm + online, cbId 异步 */
@JavascriptInterface
public void a2aQueryPro(String craft, int radiusKm, boolean online, String cbId) {
    client.queryPro(craft, radiusKm, online, new A2aClient.Callback<java.util.List<A2aClient.Peer> peers>() {
        @Override public void onSuccess(java.util.List<A2aClient.Peer> peers) {
            org.json.JSONArray arr = new org.json.JSONArray();
            try {
                for (A2aClient.Peer p : peers) {
                    arr.put(new org.json.JSONObject().put("deviceId", p.deviceId).put(
                        "name", p.name));
                }
            } catch (org.json.JSONException e) {
                evalJs("window._hermesCb('" + cbId + "',{"ok":false,"error":"serializing peers"});
                return;
            }
            evalJs("window._hermesCb('" + cbId + "',{"ok":true,"devices":arr.toString()});
        }
        @Override public void onFailure(java.io.IOException e) {
            evalJs("window._hermesCb('" + cbId + "',{"ok":false,"error":e.getMessage()});
        }
    });
}

@JavascriptInterface
public void a2aPairRequest(String cbId) {
    client.pairRequest(new A2aClient.Callback<String>() {
        @Override public void onSuccess(String code) {
            evalJs("window._hermesCb('" + cbId + "',{"ok":true,"code":code});
        }
        @Override public void onFailure(java.io.IOException e) {
            evalJs("window._hermesCb('" + cbId + "',{"ok":false,"error":e.getMessage()});
        }
    });
}

@JavascriptInterface
public void a2aPairAccept(String code, String cbId) {
    client.pairAccept(code, new A2aClient.Callback<A2aClient.Peer>() {
        @Override public void onSuccess(A2aClient.Peer p) {
            evalJs("window._hermesCb('" + cbId + "',{"ok":true,"peer":{"deviceId":p.deviceId,"name":p.name}});
        }
        @Override public void onFailure(java.io.IOException e) {
            evalJs("window._hermesCb('" + cbId + "',{"ok":false,"error":e.getMessage()});
        }
    });
}
}

```

```

@JavascriptInterface
public void a2aPeers(String cbId) {
    client.peers(new A2aClient.Callback<List<A2aClient.Peer>>>() {
        @Override public void onSuccess(List<A2aClient.Peer> peers) {
            JSONArray arr = new JSONArray();
            for (A2aClient.Peer p : peers) {
                JSONObject o = new JSONObject();
                try { o.put("deviceId", p.deviceId); o.put("name", p.name); } catch (E
                    arr.put(o);
            }
            evalJs("window._hermesCb('" + cbId + "',{\"ok\":true,\"peers\": " + arr.toS
        }
        @Override public void onFailure(java.io.IOException e) {
            evalJs("window._hermesCb('" + cbId + "',{\"ok\":false,\"error\":\"" + esc(
        }
    });
}

@JavascriptInterface
public void a2aSend(String toId, String type, String payloadJson, String cbId) {
    try {
        JSONObject payload = new JSONObject(payloadJson);
        client.send(toId, type, payload, new A2aClient.Callback<Long>() {
            @Override public void onSuccess(Long msgId) {
                auditJournal("a2a_send", new JSONObject(), toId, type, payload);
                evalJs("window._hermesCb('" + cbId + "',{\"ok\":true,\"msgId\": " + msg
            }
            @Override public void onFailure(java.io.IOException e) {
                evalJs("window._hermesCb('" + cbId + "',{\"ok\":false,\"error\":\"" +
            }
        });
    } catch (Exception e) {
        evalJs("window._hermesCb('" + cbId + "',{\"ok\":false,\"error\":\"" + esc(e.ge
    }
}

@JavascriptInterface
public void a2aStartPolling() {
    client.startPolling(msgs -> {
        for (A2aClient.Msg m : msgs) {
            auditJournal("a2a_recv", m.payload, m.fromId, m.type, m.payload);
            JSONObject ev = new JSONObject();
            try {
                ev.put("type", "a2a_msg");
                ev.put("fromId", m.fromId);
                ev.put("msgType", m.type);
                ev.put("payload", m.payload != null ? m.payload : new JSONObject());
            } catch (Exception ignored) {}
            activity.evalJsPublic("window._a2aMsg && _a2aMsg(" + ev.toString() + ")");
        }
    });
}

```

```
    }  
    });  
}  
  
@JavascriptInterface  
public void a2aStopPolling() {  
    client.stopPolling();  
}  
  
/* — 商家数据: 菜单 JSON — */  
@JavascriptInterface  
public String a2aLoadMenu() {  
    try {  
        File f = new File(dataDir, "menu.json");  
        if (!f.exists()) return "[]";  
        byte[] b = Files.readAllBytes(f.toPath());  
        return new String(b, StandardCharsets.UTF_8);  
    } catch (Exception e) { return "[]"; }  
}  
  
@JavascriptInterface  
public String a2aSaveMenu(String menuJson) {  
    try {  
        File f = new File(dataDir, "menu.json");  
        try (FileWriter w = new FileWriter(f)) { w.write(menuJson); }  
        return "{\"ok\":true}";  
    } catch (Exception e) { return "{\"ok\":false,\"error\":\"\" + esc(e.getMessage()) }  
}  
  
/* — journal 审计 — */  
private void auditJournal(String type, JSONObject payload, String peerId, String msgTy  
    try {  
        JSONObject event = new JSONObject();  
        event.put("type", type);  
        event.put("actor", "a2a");  
        event.put("taskId", JSONObject.NULL);  
        JSONObject p = new JSONObject();  
        p.put("peerId", peerId != null ? peerId : "");  
        p.put("msgType", msgType != null ? msgType : "");  
        p.put("audit", true);  
        if (msgPayload != null) p.put("msgPayload", msgPayload);  
        event.put("payload", p);  
        // 写 journal (走默认房间 'a2a')  
        com.hermes.android.agent.Journal j = new com.hermes.android.agent.Journal(  
            activity.getStorageManager().getRoomsDir());  
        j.append("a2a", event);  
    } catch (Exception e) {  
        android.util.Log.w("BridgeA2a", "audit journal: " + e.getMessage());  
    }  
}
```

```

    private static String esc(String s) {
        if (s == null) return "";
        return s.replace("\\", "\\").replace("\n", "\\n");
    }
}
BridgeAi.java
package com.hermes.android.bridge;

import com.hermes.android.HermesActivity;
import com.hermes.android.agent.AgentLoop;
import com.hermes.android.scene.DramaSource;
import com.hermes.android.agent.IntentEngine;
import com.hermes.android.ai.AiClient;
import com.hermes.android.ai.AiProviderConfig;
import com.hermes.android.ai.ContextWindow;
import com.hermes.android.model.ModelRegistry;

import org.json.JSONArray;
import org.json.JSONObject;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.ExecutorService;

/**
 * AI Bridge - 异步对话 + Council + 配置查询
 */
public class BridgeAi extends BaseBridge {

    private final AiProviderConfig aiConfig;
    private final ModelRegistry modelRegistry;
    private final ExecutorService aiExecutor;
    // ARCH_DEBT #8: per-room chatHistory, 按 roomId 分桶防串上下文
    private final java.util.Map<String, java.util.List<AiClient.Message>> roomHistories =
    private static final String GLOBAL_KEY = "__global__";
    /** 工单 17: 首页对话固定房间 (方案 b - 首页对话落 journal 房间, 全局桶从它懒重建)。
     * isValidId 允许字母数字混合 → "r_home" 合法; face_history 扫描跳过该房间 (MemoryToolProvider)。 */
    private static final String HOME_ROOM_ID = "r_home";
    private final com.hermes.android.agent.Journal journal;
    private static final int MAX_HISTORY = 50;
    /** FUSION F2: 最近一次 agentStart 构建的工具注册表 (快脑查表转发复用同一张表)。
     * volatile: agentStart 在工作线程构建后写入, 快脑 query 在 JS 桥线程读取。
     * happens-before 依据: volatile store 先于后续 volatile load 可见 (JLS §17.4.5),
     * registry 内部 (handler 等) 在 store 之前完成初始化, load 线程必看到完整构建的表。 */
    private volatile com.hermes.android.agent.ToolRegistry registry;

    /** FUSION F2: 快脑查表转发用 (可能为 null—agentStart 尚未跑过) */
    public com.hermes.android.agent.ToolRegistry getRegistry() { return registry; }

```

```

private java.util.List<AiClient.Message> getHistory(String roomId) {
    String key = roomId != null ? roomId : GLOBAL_KEY;
    java.util.List<AiClient.Message> hist = roomHistories.computeIfAbsent(key,
        k -> java.util.Collections.synchronizedList(new java.util.ArrayList<>()));
    // BUG-4b: 首次访问房间桶时从 journal.jsonl 懒重建 (防重入守卫)
    // 工单 17: 全局桶 (首页对话) 也从固定首页房间 r_home 的 journal 懒重建 (方案 b)
    String srcRoom = roomId != null ? roomId : HOME_ROOM_ID;
    if (hist.isEmpty() && !rebuilding.contains(srcRoom)) {
        rebuilding.add(srcRoom);
        try {
            rebuildFromJournal(srcRoom, hist);
        } finally {
            rebuilding.remove(srcRoom);
        }
    }
    return hist;
}

/** 工单 17: 首页对话 (全局桶) 成功后持久化到 r_home journal —
 * 事件格式对齐 rebuildHistoryFromJournal 提取规则 (user_input→user / deliver→assistant) */
private void persistHomeChat(String userText, String replyText) {
    try {
        JSONObject u = new JSONObject()
            .put("actor", "user")
            .put("type", "user_input")
            .put("taskId", JSONObject.NULL)
            .put("payload", new JSONObject().put("text", userText));
        if (journal.append(HOME_ROOM_ID, u) < 0) return;
        JSONObject a = new JSONObject()
            .put("actor", "assistant")
            .put("type", "deliver")
            .put("taskId", JSONObject.NULL)
            .put("payload", new JSONObject().put("summary", replyText));
        journal.append(HOME_ROOM_ID, a);
    } catch (Exception e) {
        android.util.Log.w("BridgeAi", "persistHomeChat: " + e.getMessage());
    }
}

/** BUG-4b: 防重入守卫 - rebuildFromJournal 不再回调 getHistory, 避免循环 */
private final java.util.Set<String> rebuilding = java.util.Collections.synchronizedSet

/** BUG-4b: 从房间 journal.jsonl 懒重建内存历史桶 (真实落盘路径) */
private void rebuildFromJournal(String roomId, java.util.List<AiClient.Message> hist)
    if (roomId == null || roomId.isEmpty()) return;
    try {
        java.io.File journalFile = new java.io.File(
            activity.getStorageManager().getRoomsDir(), roomId + "/journal.jsonl")
        rebuildHistoryFromJournal(journalFile, hist, MAX_HISTORY);
    } catch (Exception e) {

```

```

        android.util.Log.w("BridgeAi", "rebuildFromJournal: " + e.getMessage());
    }
}

/**
 * BUG-4b: 从 journal.jsonl 提取对话事件重建历史桶。返回加载条数。
 * 提取规则: user_input(actor=user) → Message("user", payload.text)
 *           deliver                → Message("assistant", payload.summary)
 *           fail                    → Message("assistant", "(失败) " + payload.reason)
 * kernel 侧 user_input (actor≠user) 不取, 防 council 主持词等系统词污染
 * 静态方法, 可独立单测 (注入临时 journal 文件)。
 */
static int rebuildHistoryFromJournal(java.io.File journalFile,
                                     java.util.List<AiClient.Message> hist,
                                     int maxHistory) {
    if (!journalFile.exists() || !journalFile.isFile()) return 0;

    java.util.Set<String> seen = new java.util.HashSet<>();
    for (AiClient.Message m : hist) {
        seen.add(m.role + ":@" + m.content);
    }
    java.util.List<AiClient.Message> loaded = new java.util.ArrayList<>();
    try {
        String content = new String(java.nio.file.Files.readAllBytes(journalFile.toPath()),
                                     java.nio.charset.StandardCharsets.UTF_8);
        for (String line : content.split("\n")) {
            line = line.trim();
            if (line.isEmpty()) continue;
            try {
                org.json.JSONObject ev = new org.json.JSONObject(line);
                String type = ev.optString("type", "");
                String role = null;
                String text = null;

                if ("user_input".equals(type)) {
                    // 只取 actor=user 的 user_input; kernel 侧注入(council 主持词等)不进桶
                    if (!"user".equals(ev.optString("actor", ""))) continue;
                    org.json.JSONObject payload = ev.optJSONObject("payload");
                    role = "user";
                    text = payload != null ? payload.optString("text", "") : "";
                } else if ("deliver".equals(type)) {
                    org.json.JSONObject payload = ev.optJSONObject("payload");
                    role = "assistant";
                    text = payload != null ? payload.optString("summary", "") : "";
                } else if ("fail".equals(type)) {
                    org.json.JSONObject payload = ev.optJSONObject("payload");
                    role = "assistant";
                    String reason = payload != null ? payload.optString("reason", "") : "";
                    text = "(失败) " + reason;
                }
            } catch (org.json.JSONException e) {
                // 忽略解析失败的行
            }
            if (role != null) {
                String key = role + ":@" + text;
                if (!seen.contains(key)) {
                    loaded.add(new AiClient.Message(role, text));
                    seen.add(key);
                }
            }
        }
    } catch (IOException e) {
        android.util.Log.w("BridgeAi", "rebuildHistoryFromJournal: " + e.getMessage());
    }
}

```

```

        if (role != null && text != null && !text.isEmpty()) {
            String key = role + "::" + text;
            if (!seen.contains(key)) {
                seen.add(key);
                loaded.add(new AiClient.Message(role, text));
            }
        }
    } catch (Exception ignored) {}
}
} catch (Exception ignored) {}
if (!loaded.isEmpty()) {
    hist.addAll(loaded);
    while (hist.size() > maxHistory * 2) hist.remove(0);
}
return loaded.size();
}

public BridgeAi(HermesActivity activity) {
    super(activity);
    this.aiConfig = activity.getAiConfig();
    this.modelRegistry = activity.getModelRegistry();
    this.aiExecutor = activity.getAiExecutor();
    this.journal = new com.hermes.android.agent.Journal(
        activity.getStorageManager().getRoomsDir());
}

public void aiChatAsync(String text, String callbackId) { aiChatAsync(text, callbackId

public void aiChatAsync(String text, String callbackId, String roomId) {
    if (text == null || text.trim().isEmpty()) {
        evalJs("window._hermesCb('" + callbackId + "',{\\"ok\\":false,\\"content\\":\\"消息为空\\"}");
        return;
    }
    if (text.length() > 4000) {
        evalJs("window._hermesCb('" + callbackId + "',{\\"ok\\":false,\\"content\\":\\"消息过长\\"}");
        return;
    }
    aiExecutor.execute(() -> {
        String resultJson;
        try {
            String rid = (roomId != null && !roomId.isEmpty()) ? roomId : null;
            if (!aiConfig.isAiEnabled()) {
                resultJson = "{\\"ok\\":false,\\"content\\":\\"AI 已关闭, 点右上角 ≡ 可启用。\\"}";
            } else if (!aiConfig.isConfigured()) {
                // BUGFIX: 旧 AiProviderConfig 无 key 时 fallback ModelRegistry 默认模型
                com.hermes.android.model.ModelConfig dm = modelRegistry.getDefault();
                if (dm != null && dm.apiKey != null && !dm.apiKey.isEmpty()) {
                    AiClient client = new AiClient(dm);
                    java.util.List<AiClient.Message> history;

```

```

        synchronized (getHistory(rid)) {
            history = new ArrayList<>(getHistory(rid));
        }
        List<AiClient.Message> trimmed = ContextWindow.trim(history);
        AiClient.AiResponse resp = client.chat(text, trimmed);
        if (resp.success) {
            synchronized (getHistory(rid)) {
                getHistory(rid).add(new AiClient.Message("user", text));
                getHistory(rid).add(new AiClient.Message("assistant", resp
                while (getHistory(rid).size() > MAX_HISTORY * 2) getHistor
            }
            if (rid == null) persistHomeChat(text, resp.content); // 工单 1
            activity.saveChatHistoryPublic();
            resultJson = new JSONObject()
                .put("ok", true)
                .put("content", resp.content).toString();
        } else {
            resultJson = new JSONObject()
                .put("ok", false)
                .put("content", "AI 调用失败: " + resp.content).toString();
        }
    } else {
        resultJson = "{\"ok\":false,\"content\":\"AI 尚未配置 API Key, 点右上角 ≡
    }
} else {
    AiClient client = new AiClient(aiConfig);
    java.util.List<AiClient.Message> history;
    synchronized (getHistory(rid)) {
        history = new ArrayList<>(getHistory(rid));
    }
    List<AiClient.Message> trimmed = ContextWindow.trim(history);
    AiClient.AiResponse resp = client.chat(text, trimmed);
    if (resp.success) {
        synchronized (getHistory(rid)) {
            getHistory(rid).add(new AiClient.Message("user", text));
            getHistory(rid).add(new AiClient.Message("assistant", resp.con
            while (getHistory(rid).size() > MAX_HISTORY * 2) getHistory(ri
        }
        if (rid == null) persistHomeChat(text, resp.content); // 工单 17: 首
        activity.saveChatHistoryPublic();
        resultJson = new JSONObject()
            .put("ok", true)
            .put("content", resp.content).toString();
    } else {
        resultJson = new JSONObject()
            .put("ok", false)
            .put("content", "AI 调用失败: " + resp.content).toString();
    }
}
} catch (Exception e) {

```

```

        try {
            resultJson = new JSONObject()
                .put("ok", false)
                .put("content", "AI 调用异常: " + e.getMessage()).toString();
        } catch (Exception ex) {
            resultJson = "{\\"ok\\":false,\\"content\\":\\"AI 调用异常\\"}";
        }
    }
    evalJs("window._hermesCb('" + callbackId + "'," + resultJson + ")");
});
}

/** Phase 2A.2: 异步意图分类（照 aiChatAsync cbId 模式）。 */
public void intentClassifyAsync(String text, String callbackId) {
    if (text == null || text.trim().isEmpty()) {
        evalJs("window._hermesCb('" + callbackId + "',{\\"ok\\":false,\\"error\\":{\\"code\\"return;
    }
    aiExecutor.execute(() -> {
        String resultJson;
        try {
            IntentEngine engine = new IntentEngine(activity);
            com.hermes.android.agent.ToolRegistry reg = getRegistry();
            if (reg != null) engine.setToolRegistry(reg); // FUSION F4: 意图 prompt 拼注
            IntentEngine.IntentResult r = engine.classify(text);
            resultJson = new JSONObject()
                .put("ok", true)
                .put("data", new JSONObject()
                    .put("intent", r.intent)
                    .put("type", r.type)
                    .put("domain", r.domain)
                    .put("slots", new JSONObject(r.slots))
                    .put("confidence", r.confidence))
                .toString();
        } catch (Exception e) {
            try {
                resultJson = new JSONObject()
                    .put("ok", false)
                    .put("error", new JSONObject()
                        .put("code", "CLASSIFY_FAILED")
                        .put("msg", e.getMessage()))
                    .toString();
            } catch (Exception ex) {
                resultJson = "{\\"ok\\":false,\\"error\\":{\\"code\\":\\"INTERNAL\\",\\"msg\\":\\"
            }
        }
        evalJs("window._hermesCb('" + callbackId + "'," + resultJson + ")");
    });
}
}

```

```

/** Phase 2C.2: 异步追剧搜索 (照 aiChatAsync cbId 模式, aiExecutor 跑 DramaSource)。 */
public void faceSearchDramaAsync(String query, String callbackId) {
    if (query == null || query.trim().isEmpty()) {
        evalJs("window._hermesCb('" + callbackId + "', {\\"ok\\":false,\\\"error\\":{\\"code\\":\\"SEARCH_FAILED\\"}})");
        return;
    }
    aiExecutor.execute(() -> {
        String resultJson;
        try {
            JSONArray arr = new JSONArray();
            // 本地视频优先 (有权限时)
            com.hermes.android.scene.LocalVideoLibrary localLib = new com.hermes.android.scene.LocalVideoLibrary();
            if (localLib.hasPermission()) {
                java.util.List<com.hermes.android.scene.LocalVideoLibrary.LocalVideo> locals = localLib.getLocalVideos();
                for (com.hermes.android.scene.LocalVideoLibrary.LocalVideo lv : locals) {
                    arr.put(lv.toJSON());
                }
            }
            DramaSource ds = new DramaSource();
            ds.setRegistry(new com.hermes.android.agent.VideoSourceRegistry(activity));
            java.util.List<DramaSource.DramaItem> items = ds.search(query);
            for (DramaSource.DramaItem item : items) arr.put(item.toJSON());
            resultJson = new JSONObject()
                .put("ok", true)
                .put("data", new JSONObject().put("items", arr))
                .toString();
        } catch (Exception e) {
            try {
                resultJson = new JSONObject()
                    .put("ok", false)
                    .put("error", new JSONObject()
                        .put("code", "SEARCH_FAILED")
                        .put("msg", e.getMessage()))
                    .toString();
            } catch (Exception ex) {
                resultJson = "{\"ok\":false,\\\"error\\\":{\\"code\\":\\"INTERNAL\\",\\"msg\\":\\""}}";
            }
        }
        evalJs("window._hermesCb('" + callbackId + "', " + resultJson + ")");
    });
}

/** DESIGN_NEW_ROOM v2: 单聊房按房间绑定模型对话 (modelId 空/失效 → 注册表默认模型) */
public void aiChatWithModel(String text, String modelId, String callbackId) { aiChatWithModel(text, modelId, callbackId, null); }

public void aiChatWithModel(String text, String modelId, String callbackId, String roomId) {
    String rid = (roomId != null && !roomId.isEmpty()) ? roomId : null;
    if (text == null || text.trim().isEmpty()) {
        evalJs("window._hermesCb('" + callbackId + "', {\\"ok\\":false,\\\"content\\":\\"消息为空\\"})");
        return;
    }
}

```

```
}
if (text.length() > 4000) {
    evalJs("window._hermesCb('" + callbackId + "',{\\"ok\\":false,\\"content\\":\\"消息过长\\"}");
    return;
}
aiExecutor.execute(() -> {
    String resultJson;
    try {
        com.hermes.android.model.ModelConfig mc = null;
        if (modelId != null && !modelId.isEmpty()) mc = modelRegistry.get(modelId)
        if (mc == null) mc = modelRegistry.getDefault();
        if (mc == null) {
            resultJson = "{\\"ok\\":false,\\"content\\":\\"AI 尚未配置模型，运行页添加后即可畅聊。\\"}";
        } else {
            AiClient client = new AiClient(mc);
            java.util.List<AiClient.Message> history;
            synchronized (getHistory(rid)) {
                history = new ArrayList<>(getHistory(rid));
            }
            List<AiClient.Message> trimmed = ContextWindow.trim(history);
            AiClient.AiResponse resp = client.chat(text, trimmed);
            if (resp.success) {
                synchronized (getHistory(rid)) {
                    getHistory(rid).add(new AiClient.Message("user", text));
                    getHistory(rid).add(new AiClient.Message("assistant", resp.content));
                    while (getHistory(rid).size() > MAX_HISTORY * 2) getHistory(rid).remove(0);
                }
                if (rid == null) persistHomeChat(text, resp.content); // 工单 17: 首页
                activity.saveChatHistoryPublic();
                resultJson = new JSONObject()
                    .put("ok", true)
                    .put("content", resp.content).toString();
            } else {
                resultJson = new JSONObject()
                    .put("ok", false)
                    .put("content", "AI 调用失败: " + resp.content).toString();
            }
        }
    } catch (Exception e) {
        try {
            resultJson = new JSONObject()
                .put("ok", false)
                .put("content", "AI 调用异常: " + e.getMessage()).toString();
        } catch (Exception ex) {
            resultJson = "{\\"ok\\":false,\\"content\\":\\"AI 调用异常\\"}";
        }
    }
    evalJs("window._hermesCb('" + callbackId + "'," + resultJson + ")");
});
}
```

```

function settingsBack() {
    if (_setStack.length) { _setCur = _setStack.pop(); renderSettingsPage(_setCur); }
    else closeSettings();
}

function renderSettingsPage(id) {
    var pane = ensureSettingsPane();
    if (!pane) return;
    var html = '';
    if (id === 'main') html = renderSettingsTree();
    else html = renderSubPage(id);
    pane.innerHTML = html;
    /* 二级页异步数据（保险柜/设备协作/系统状态）渲染后填充 */
    if (id !== 'main') setTimeout(function(){ try { afterSubPage(id); } catch(e){} }, 30);
}

function afterSubPage(id) {
    if (id === 'vault') loadVaultEntries();
    else if (id === 'device') refreshA2aBlock();
    else if (id === 'system') _renderSysStatus();
    else if (id === 'applyment' && window.MovApplyment) MovApplyment.after();
    else if (id === 'account' && window.MovApplyment) MovApplyment.after();
    else if (id === 'verify' && window.MovApplyment) MovApplyment.after();
    else if (id === 'ability' && window.MovGig) MovGig.after();
    else if (id === 'market' && window.MovMarket && window.MovMarket.bind) window.MovMarke
}

function settingsTopbar(title) {
    return '<div class="set-topbar"><span class="set-back" onclick="NewFace.settingsBack()'
        + '<span class="set-ttl">' + escHtml(title) + '</span></div>';
}

/* — 二级页分发 — */
var _SUB_TITLES = {
    account:'账户管理', market:'MCP 市场', merchant:'商户收款', verify:'身份认证', ability:'我的能力',
    theme:'外观', model:'AI 模型', vault:'信息管理', feedback:'更新反馈', adv:'高级',
    understand:'理解方式', device:'设备协作', scene:'快捷场景', perm:'权限与功能',
    service:'服务连接', system:'系统', applyment:'特约商户进件'
};

function renderSubPage(id) {
    var title = _SUB_TITLES[id] || '设置';
    var body = '';
    if (id === 'merchant') body = renderMerchantPage();
    else if (id === 'market') body = (window.MovMarket && window.MovMarket.render) ? windo
    else if (id === 'theme') body = renderThemePage();
    else if (id === 'model') body = renderModelPage();
    else if (id === 'vault') body = renderVaultPage();
    else if (id === 'feedback') body = renderFeedbackPage();
    else if (id === 'adv') body = renderAdvPage();
}

```

```

else if (id === 'understand') body = renderUnderstandPage();
else if (id === 'device') body = renderDevicePage();
else if (id === 'scene') body = renderScenePage();
else if (id === 'perm') body = renderPermPage();
else if (id === 'service') body = renderServicePage();
else if (id === 'system') body = renderSystemPage();
else if (id === 'applyment') body = window.MovApplyment ? MovApplyment.renderForm() :
else if (id === 'account') body = window.MovApplyment ? MovApplyment.renderAccount() :
else if (id === 'verify') body = window.MovApplyment ? MovApplyment.renderVerify() :
else if (id === 'ability') body = (window.MovGig && window.MovGig.renderAbility) ? win
return settingsTopbar(title) + '<div class="set-page">' + body + '</div>';
}

/* 商户收款二级页（工单 26 M0-1）：四证 + AppID + 卖家签名私钥 → SecureVault, prefs 只存 vault id + appid 明文
*/
function renderMerchantPage() {
return '<div class="set-group">'
+ '<div class="set-sub" style="padding-top:12px">商户四证（微信支付直连）</div>'
+ '<input id="mkMchid" placeholder="商户号 mchid">'
+ '<input id="mkAppid" placeholder="公众号/应用 AppID（非机密，明文存）">'
+ '<input id="mkSerial" placeholder="API 证书序列号">'
+ '<input id="mkApiv3" type="password" placeholder="APIv3 密钥">'
+ '<textarea id="mkPem" rows="5" placeholder="API 证书私钥 pem（整段粘贴）" style="width:100%;>'
+ '<div class="set-sub" style="padding-top:12px">卖家发货签名（可选，卖家身份发货用）</div>'
+ '<textarea id="mkSellerPem" rows="4" placeholder="卖家 RSA 私钥 pem（整段粘贴）" style="width:'
+ '<div class="note">私钥只进本机保险柜（SecureVault 加密存储），不落 journal / 日志 / 明文存储。</div>'
+ '<div class="mk-buybar" style="margin-top:10px"><span class="dact" onclick="NewFac
+ '</div>';
}

/* 保存四证 + AppID + 卖家私钥（cbId 异步桥 mktSaveMerchant） */
function merchantSave() {
var mchid = v('mkMchid'), appid = v('mkAppid'), serial = v('mkSerial'), apiv3 = v('mkA
pem = v('mkPem'), sellerPem = v('mkSellerPem');
if (!mchid || !appid || !serial || !apiv3 || !pem) { B.toast('请填写全商户四证 + AppID'); retur
if (!window.HermesBridge || !window.HermesBridge.mktSaveMerchant) { B.toast('市场桥未就绪');
var id = (typeof nextCbId === 'function') ? nextCbId() : ('cb' + Date.now());
if (typeof _cbMap !== 'undefined') _cbMap[id] = function(r) {
if (r && r.ok) { B.toast('已存入保险柜'); emptyMerchantInputs(); }
else B.toast((r && r.error && r.error.msg) ? ('保存失败: ' + r.error.msg) : '保存失败');
};
window.HermesBridge.mktSaveMerchant(JSON.stringify({
mchid: mchid, appid: appid, serial: serial, apiv3Key: apiv3, pem: pem, sellerPem: se
}), id);
}
function v(id) { var el = document.getElementById(id); return el ? el.value.trim() : '';
function emptyMerchantInputs() {
['mkMchid', 'mkAppid', 'mkSerial', 'mkApiv3', 'mkPem', 'mkSellerPem'].forEach(function(id)
var el = document.getElementById(id); if (el) el.value = '';
});
}
}

```

```

/* 高级二级页（工单 25：原一级页折叠组改二级页，六行入口不变） */
function renderAdvPage() {
    return '<div class="set-group">'
        + '<div class="set-row" onclick="NewFace.settingsGo(\`understand\`)"><div class="t">'
        + '<div class="set-row" onclick="NewFace.settingsGo(\`device\`)"><div class="t">设备协作'
        + '<div class="set-row" onclick="NewFace.settingsGo(\`scene\`)"><div class="t">快捷场景<'
        + '<div class="set-row" onclick="NewFace.settingsGo(\`perm\`)"><div class="t">权限与功能<'
        + '<div class="set-row" onclick="NewFace.settingsGo(\`service\`)"><div class="t">服务连'
        + '<div class="set-row" onclick="NewFace.settingsGo(\`system\`)"><div class="t">系统</'
        + '</div>';
}

/* 外观二级页：浅色/深色/跟随系统 打勾单选（mov_theme: light/dark/auto） */
function renderThemePage() {
    var cur = getThemeMode();
    var opts = [['light', '浅色'], ['dark', '深色'], ['auto', '跟随系统']];
    var out = '<div class="set-group">';
    opts.forEach(function(o) {
        out += '<div class="set-row" onclick="NewFace.setThemeMode(\`' + o[0] + '\`)"><div c'
            + '<div class="check" + (cur === o[0] ? ' : ' ghost') + ">✓</div></div>';
    });
    out += '</div>';
    return out;
}

/* 更新反馈二级页（工单 25：检查更新行 + 原意见反馈表单合并） */
function renderFeedbackPage() {
    return '<div class="set-group">'
        + '<div class="set-row" onclick="NewFace.checkUpdate()"><div class="t">检查更新</div><di'
        + '</div>'
        + '<div class="set-group fb-group"><textarea id="fbText" placeholder="说说哪里不好用..."></te'
        + '<div class="set-group">'
        + '<div class="set-row static"><div class="t">附带诊断日志</div>'
        + '<div class="switch on" id="fbDiag" onclick="NewFace._fbToggleDiag()"></div></div>'
        + '</div>'
        + '<div class="note">日志只含运行状态，不含对话内容和密钥。</div>'
        + '<div class="set-group"><div class="add-model" onclick="NewFace._fbSubmit()">提交</d'
}

function _fbToggleDiag() {
    var sw = document.getElementById('fbDiag');
    if (sw) sw.classList.toggle('on');
}

function _fbSubmit() {
    var t = document.getElementById('fbText');
    var txt = (t && t.value) ? t.value.trim() : '';
    if (!txt) { B.toast('先写点反馈内容'); return; }
    var sw = document.getElementById('fbDiag');
    if (sw && sw.classList.contains('on')) {
        try { B.postCmd({cmd: 'diagnostics.export'}); } catch(e) {}
    }
}

```

```

    }
    B.toast('已提交, 感谢反馈');
    redrawSettings();
}

/* 理解方式二级页: 更准/更快 单选, 各带代价说明 (原意图路由, localStorage mv:routing 兼容) */
function renderUnderstandPage() {
    var cur = getRoutingMode();
    var opts = [
        ['llm', '更准', '先让 AI 理解你的意思再行动, 复杂任务更可靠, 用量略高'],
        ['regex', '更快', '常用指令直接执行, 响应快、省用量']
    ];
    var out = '<div class="set-group">';
    opts.forEach(function(o) {
        out += '<div class="set-row" onclick="NewFace.setRoutingMode(\'' + o[0] + '\')"><div'
            + '<div class="sub">' + o[2] + '</div></div>'
            + '<div class="check" + (cur === o[0] ? ' : ' ghost') + '">✓</div></div>';
    });
    out += '</div>';
    return out;
}

/* 检查更新: 读 MOV_APP_VERSION 展示 (无网络更新逻辑, 桥零新增) */
function checkUpdate() {
    B.toast('当前已是最新版本 v' + MOV_APP_VERSION);
}

/** 一级页 (工单 25 定稿): 账户管理 → MCP 市场 → AI 模型 → 信息管理 → 高级 → 外观 → 更新反馈 */
function renderSettingsTree() {
    var html = settingsTopbar('设置');
    html += '<div class="set-page">';
    html += '<div class="set-group" style="margin-top:6px">'
        + '<div class="set-row" onclick="NewFace.settingsGo(\''account\')"><div class="t">账户管'
        + '</div>';
    html += '<div class="set-group">'
        + '<div class="set-row" onclick="NewFace.settingsGo(\''market\')"><div class="t">MCP'
        + '</div>';
    html += '<div class="set-group">'
        + '<div class="set-row" onclick="NewFace.settingsGo(\''model\')"><div class="t">AI 模型'
        + '</div>';
    html += '<div class="set-group">'
        + '<div class="set-row" onclick="NewFace.settingsGo(\''account\')"><div class="t">账户管'
        + '<div class="set-row" onclick="NewFace.settingsGo(\''vault\')"><div class="t">信息管理<'
        + '</div>';
    html += '<div class="set-group">'
        + '<div class="set-row" onclick="NewFace.settingsGo(\''adv\')"><div class="t">高级</div'
        + '</div>';
    html += '<div class="set-group">'
        + '<div class="set-row" onclick="NewFace.settingsGo(\''theme\')"><div class="t">外观</d'
        + '</div>';
}

```

```

    html += '<div class="set-group">'
    + '<div class="set-row" onclick="NewFace.settingsGo(\'feedback\')"><div class="t">更新'
    + '</div>';
    html += '</div>';
    return html;
}

/* — 一级页 summary 值（状态收进二级页，一级只露名称/简短值） — */

function themeSummary() {
    var m = getThemeMode();
    return m === 'dark' ? '深色' : (m === 'auto' ? '跟随系统' : '浅色');
}

function modelSummary() {
    try {
        var models = (typeof B !== 'undefined' && B.listModels) ? B.listModels() : [];
        for (var i = 0; i < models.length; i++) if (models[i].isDefault) return (models[i].n
        return models.length ? (models[0].name || '模型') : '未配置';
    } catch(e) { return '未配置'; }
}

function understandSummary() {
    return getRoutingMode() === 'regex' ? '更快' : '更准';
}

function deviceSummary() {
    if (_a2aPeerCache && _a2aPeerCache.length) return _a2aPeerCache.length + ' 台已配对';
    return '未连接';
}

function sceneSummary() {
    var n = 0;
    try {
        var resp = (typeof B !== 'undefined' && B.query) ? B.query({q:'tool_manifest'}) : nu
        var tools = (resp && resp.ok && resp.data && resp.data.tools) ? resp.data.tools : []
        for (var i = 0; i < tools.length; i++) if (tools[i].level === 'FAST') n++;
    } catch(e) {}
    return n ? n + ' 个可用' : '暂无';
}

function permSummary() {
    var n = 0;
    try {
        var perms = (typeof B !== 'undefined' && B.permState) ? B.permState() : {};
        var names = {CAMERA:1, LOCATION:1, CONTACTS:1, SMS:1, CALL:1, NOTIFY:1, SETTINGS:1};
        for (var k in perms) { if (names[k] && !perms[k]) n++; }
    } catch(e) {}
    return n ? n + ' 项待开启' : '已全部开启';
}

```

```
function serviceSummary() {
  var on = [];
  try {
    var resp = (typeof B !== 'undefined' && B.query) ? B.query({q:'web_sites'}) : null;
    var sites = (resp && resp.ok && resp.data && resp.data.sites) ? resp.data.sites : []
    sites.forEach(function(s) {
      if (!(s.auth && s.auth.loginUrl)) return;
      var id = s.site.replace('.com','');
      try { if ((typeof B !== 'undefined' && B.connectStatus) && B.connectStatus(id).con
    });
  } catch(e) {}
  return on.length ? on.join(' · ') : '未登录';
}
```

```
function refreshA2aCache() {
  try {
    if (typeof B !== 'undefined' && B.a2aPeers) {
      B.a2aPeers(function(peers) {
        _a2aPeerCache = (peers && peers.length) ? peers : [];
        if (_setCur === 'main') renderSettingsPage('main');
      });
    }
  } catch(e) {}
}
```

/* — AI 模型二级页（工单 14：模型卡 + 添加 + 本月用量条 + 本地模型底部） — */

```
var _mState = {opsId:null, formOpen:false, editId:null, presets:[], provider:'deepseek',
function _mfv(id){ var e = document.getElementById(id); return e ? e.value.trim() : '';
```

```
function renderModelPage() {
  var out = '';
  /* 模型卡列表（测连接/编辑/设为默认/删除，删除过确认 sheet） */
  try {
    var models = (typeof B !== 'undefined' && B.listModels) ? B.listModels() : [];
    models.forEach(function(m){
      var defTag = m.isDefault ? '<span class="deftag"> · 默认</span>' : '';
      out += '<div class="model-card">'
      + '<div class="mname">' + escHtml(m.name || _pvName(m.provider)) + defTag + '</d'
      + '<div class="msub">' + escHtml(modelStatusText(m)) + '</div>'
      + '<div class="macts">'
      + '<div class="btn" onclick="NewFace._mtest(\'' + escHtml(m.id) + '\')">测连接</div>'
      + '<div class="btn" onclick="NewFace._medit(\'' + escHtml(m.id) + '\')">编辑</div>'
      + (m.isDefault ? '' : '<div class="btn" onclick="NewFace._setDefault(\'' + escH'
      + '<div class="btn" onclick="NewFace._delete(\'' + escHtml(m.id) + '\')">删除</di'
      + '</div></div>';
    });
    if (!models.length) {
      out += '<div class="model-card"><div class="mname">未配置模型</div>'
      + '<div class="msub">先添加一个模型才能开始派活</div></div>';
    }
  }
}
```

```

    }
    out += '<div class="set-group"><div class="add-model" onclick="NewFace._mform()">+ 添
    if (_mState.formOpen) out += _mformHtml();
  } catch(e) {}
  /* 本月用量约 N% (原 TOKEN 仪表盘收进此处, 超 80% 才上浮一级警告) */
  try {
    var ts = (typeof B !== 'undefined' && B.tokenStats) ? B.tokenStats() : {today:0, mont
    var pct = (ts.quota > 0) ? Math.min(100, Math.round((ts.month || 0) / ts.quota * 100
    out += '<div class="set-group"><div class="usage">本月用量约 ' + pct + '%'
    + '<div class="dsub">今日 ' + NewFaceExpert.tokFmt(ts.today) + ' · 本月 ' + NewFaceExp
    + '</div><div class="ubar"><i style="width:' + pct + '%"></i></div></div>';
  } catch(e) {}
  /* 本地模型 (OCR, 施工决策: 放 AI 模型页底部, 用户按「模型」找得到) */
  try {
    var lm = (typeof B !== 'undefined' && B.listLocalModels) ? B.listLocalModels() : {ok
    var lmodels = (lm && lm.models) ? lm.models : [];
    if (lmodels.length) {
      lmodels.forEach(function(m) {
        out += '<div class="set-group"><div class="set-row" onclick="NewFace.deleteLocal
        + '<div class="v">' + escHtml(m.name) + ' · ' + (m.sizeMB || 0) + ' MB · 点按删除<
      });
    } else {
      out += '<div class="set-group"><div class="set-row static"><div class="t">OCR 模型</
      + '<div class="v">未下载 · 约 2GB (识别图片/文档文字)</div></div></div>';
    }
  } catch(e) {}
  out += '<div class="note">API Key 只存本机, 支持 DeepSeek / Kimi / 通义 / OpenAI 兼容接口.</div>';
  return out;
}

/* 添加/编辑表单 (二级页内联) */
function _mformHtml() {
  var opts = '';
  _mState.presets.forEach(function(p){ opts += '<option value="' + escHtml(p.key) + '"'
  if (!opts) opts = '<option value="">无预设</option>';
  return '<div class="mform">'
  + '<div class="mform-label">厂商</div><select id="mfProvider" onchange="NewFace._mfPro
  + '<div class="mform-label">API Key</div><input id="mfKey" placeholder="sk-..." valu
  + '<div class="mform-label">Base URL</div><input id="mfBase" placeholder="https://..
  + '<div class="mform-label">模型名</div><input id="mfModel" placeholder="gpt-4o" value=
  + '<div class="mform-label">显示名</div><input id="mfName" placeholder="我的模型" value=""
  + '<div class="mform-acts">'
  + '<div class="ditem pulse" onclick="NewFace._msave()">保存</div>'
  + '<div class="ditem" onclick="NewFace._mtestForm()">测连接</div>'
  + '<div class="ditem" onclick="NewFace._mformCancel()">取消</div></div></div>';
}
function _mops(id) { _mState.opsId = (id === _mState.opsId) ? null : id; _mState.formOpe
function _mopsClose() { _mState.opsId = null; redrawSettings(); }
function _msetDefault(id) { try { if (B.setDefaultModel) B.setDefaultModel(id); B.toast(
function _mtest(id) {

```

```

    try {
        var m = B.listModels().find(function(x){ return x.id === id; });
        if (!m) return;
        B.toast('测连接中...');
        B.aiChatWithModel('只回复两个字：收到', id, function(res){ B.toast(res && res.ok ? '连接正常' :
    } catch(e){}
}
function _medit(id) {
    try {
        var m = B.listModels().find(function(x){ return x.id === id; });
        if (!m) return;
        _mState.editId = id; _mState.formOpen = true; _mState.opsId = null;
        _mState.provider = m.provider || 'deepseek';
        _mState.fields = {key:'', base:m.baseUrl || '', model:m.model || '', name:m.name ||
        redrawSettings();
    } catch(e){}
}
function _mdelete(id) {
    /* 删除模型 = 破坏性操作，过确认 sheet（红线） */
    try {
        var m = B.listModels().find(function(x){ return x.id === id; });
        var label = m ? (m.name || '该模型') : '该模型';
        expertConfirm(null, '删除模型「' + esc(label) + '」？删除后不可恢复。', function(){
            try { if (B.deleteModel) B.deleteModel(id); B.toast('已删除'); } catch(e){ B.toast('删
                _mState.opsId = null; redrawSettings();
            });
        } catch(e){}
    }
}
function _mform() {
    if (_mState.presets.length === 0) try { _mState.presets = B.providerPresets() || []; }
    _mState.formOpen = !_mState.formOpen; _mState.opsId = null; _mState.editId = null;
    if (_mState.formOpen) _mState.fields = {key:'', base:'', model:'', name:''};
    redrawSettings();
}
function _mfProvider(p) {
    _mState.fields = {key:_mfv('mfKey'), base:_mfv('mfBase'), model:_mfv('mfModel'), name:
    _mState.provider = p || 'deepseek';
    redrawSettings();
}
function _mformCancel() { _mState.formOpen = false; _mState.editId = null; redrawSetting
function _collectMForm() {
    var pres = null;
    _mState.presets.forEach(function(p){ if (p.key === _mState.provider) pres = p; });
    return {
        provider: _mState.provider,
        apiKey: _mfv('mfKey'),
        baseUrl: _mfv('mfBase'),
        model: _mfv('mfModel'),
        name: _mfv('mfName') || (pres ? pres.displayName : _mState.provider),
        role: '通用',
    }
}

```

```

        color: _pvColor(_mState.provider),
        enabled: true,
        isDefault: false
    });
}
function _msave() {
    try {
        var o = _collectMForm();
        /* 工单 13: 新建模型必须填 key(与测连接同校验); 编辑时 key 留空=不改(服务端 mergeOnUpdate 保留) */
        if (!_mState.editId && _mState.provider !== 'ollama' && !o.apiKey) { B.toast('先填 API
        if (_mState.editId) o.id = _mState.editId;
        var res = _mState.editId ? B.updateModel(o) : B.addModel(o);
        if (res && res.ok) { B.toast('已保存'); _mState.formOpen = false; _mState.editId = null
        else B.toast((res && res.error) || '保存失败');
        } catch(e){ B.toast('保存失败'); }
    }
}
function _mtestForm() {
    try {
        var o = _collectMForm();
        if (_mState.provider !== 'ollama' && !o.apiKey) { B.toast('先填 API Key'); return; }
        B.toast('测连接中...');
        B.testModel(o, function(res){ B.toast(res && res.ok ? '连接正常' : '连接失败' + (res && res.
        } catch(e){}
    }
}
function redrawSettings() { renderSettingsPage(_setCur); }

/** 删除本地模型文件（AI 模型二级页点按） */
function deleteLocalModel(name) {
    try { if (typeof B !== 'undefined' && B.deleteLocalModel) B.deleteLocalModel(name); }
    redrawSettings();
}

function getRoutingMode() {
    try { return localStorage.getItem('mv:routing') === 'regex' ? 'regex' : 'llm'; } catch
}
/** 理解方式: 更准 = llm 优先, 更快 = 仅快速通道（原意图路由, 文案重做） */
function setRoutingMode(mode) {
    try { localStorage.setItem('mv:routing', mode === 'regex' ? 'regex' : 'llm'); } catch(
    redrawSettings();
}

/* — 服务连接二级页（工单 14: 点名服务行, 原「连接」区块收此） — */

/** 从 web_sites 注册表读可连接站点（有 auth 配置的），兜底 douyin/tencent */
function connectPlatforms() {
    var platforms = [];
    try {
        var resp = (typeof B !== 'undefined' && B.query) ? B.query({q:'web_sites'}) : null;
        var sites = (resp && resp.ok && resp.data && resp.data.sites) ? resp.data.sites : []
        sites.forEach(function(s) {

```

```

        if (s.auth && s.auth.loginUrl) platforms.push({id: s.site.replace('.com', ''), label: s.site, sub: s.auth.loginUrl});
    } catch(e) {}
    if (!platforms.length) platforms = [
        {id: 'douyin', label: '抖音', sub: '搜剧看片免登录 · 会话只存本机'},
        {id: 'tencent', label: '腾讯视频', sub: 'H5 搜索免登录 · 直接看片'}
    ];
    return platforms;
}

function renderServicePage() {
    var out = '<div class="set-group">';
    connectPlatforms().forEach(function(p) {
        var connected = false;
        try { if (typeof B !== 'undefined' && B.connectStatus) connected = !!B.connectStatus }
        out += '<div class="set-row" onclick="NewFace.openConnect(\'' + escHtml(p.id) + '\')'
            + '<div class="v">' + (connected ? '已连接' : '未登录') + '</div><div class="chev">></div>';
        if (connected) {
            out += '<div class="set-sub"><span class="dact" onclick="NewFace.clearConnect(\''
                + '<span class="dact" onclick="NewFace.openConnect(\'' + escHtml(p.id) + '\')">去
            }
        }
    });
    out += '</div>';
    out += '<div class="note">登录态保存在本机，用于订票、点外卖等真实下单场景。</div>';
    return out;
}

/** 清除平台登态 (CookieManager 关键键过期) 并刷新设置页 */
function clearConnect(platform) {
    try { if (typeof B !== 'undefined' && B.removeCookies) B.removeCookies(platform); } catch(e) {}
    redrawSettings();
}

/* — 快捷场景二级页 (工单 14: FAST 卡清单 + 视频源收编底部) — */

function renderScenePage() {
    var out = '<div class="set-group">';
    try {
        var resp = (typeof B !== 'undefined' && B.query) ? B.query({q: 'tool_manifest'}) : null;
        var tools = (resp && resp.ok && resp.data && resp.data.tools) ? resp.data.tools : [];
        var any = false;
        for (var i = 0; i < tools.length; i++) {
            if (tools[i].level !== 'FAST') continue;
            any = true;
            var t = tools[i];
            var ex = (t.examples && t.examples.length) ? '例: ' + t.examples[0] : '';
            out += '<div class="set-row static"><div class="t">' + escHtml(t.name) + '</div>'
                + '<div class="v">' + escHtml(t.toolset) + '</div></div>'
                + (ex ? '<div class="set-sub">' + escHtml(ex) + '</div>' : '');
        }
    }
}

```

```

    if (!any) out += '<div class="set-row static"><div class="t">暂无可场景</div>'
    + '<div class="v">先运行一次任务，注册表加载后自动出现</div></div>';
  } catch(e) {}
  out += '</div>';
  out += '<div class="note">快捷场景是一句话直达的服务入口，在首页直接说出需求即可触发，无需在此配置。</div>';
  /* 视频源开关（施工决策：原场景卡区块内容，收编本页底部，不丢功能） */
  out += renderSourcesSection();
  return out;
}

```

/* — 设备协作二级页（工单 14：配对码/已配对设备/Relay；原 A2A 区块收此） — */

```

function renderDevicePage() {
  var out = '<div class="set-group">';
  /* 状态 + 连接另一台设备（配对码） */
  out += '<div class="set-row static"><div class="t">状态</div><div class="v">' + escHtml(
  out += '<div class="set-row" onclick="NewFace.genA2aPairCode()"><div class="t">连接另一台设备
    + '<div class="v" id="a2aMyCode">生成配对码</div><div class="chev">></div></div>';
  /* 输入配对码（自对话流 renderA2aPeerList 收编） */
  out += '<div class="set-sub"><input id="a2aPairInput" class="sheet-input flex lg" plac
    + '<button class="a2a-btn small" onclick="NewFace._a2aPair()">连接</button></div>';
  /* 已配对设备列表（异步） */
  out += '<div class="set-sub"><div class="set-mini-label">已配对设备</div>'
    + '<div id="a2aPeerList" class="dsub meta pad6">加载中...</div></div>';
  /* relay 服务器地址编辑 */
  out += '<div class="set-sub"><div class="set-mini-label">Relay 服务器</div>'
    + '<div class="f-gap"><input id="a2aRelayInput" class="sheet-input flex" placeholder
    + '<button class="a2a-btn small" onclick="NewFace.saveA2aRelay()">保存</button></div><
  out += '</div>';
  out += '<div class="note">连接后两台设备可在同一个房间里协作，任务可互相转交。</div>';
  /* 异步填充：设备列表 + relay 当前值（渲染后 DOM 就绪） */
  setTimeout(function() { refreshA2aBlock(); }, 50);
  return out;
}

```

/** 生成我的配对码（a2aPairRequest 异步回调写入） */

```

function genA2aPairCode() {
  try {
    if (typeof B !== 'undefined' && B.a2aPairRequest) {
      B.a2aPairRequest(function(r) {
        var el = document.getElementById('a2aMyCode');
        if (el && r && r.pairCode) el.innerHTML = '配对码: <b class="pair-code">' + escHtml
          else if (el) el.innerHTML = '生成失败，请重试';
      });
    }
  } catch(e) {}
}

```

/** 异步填充已配对设备列表 + relay 当前值 */

```
function refreshA2aBlock() {
```

```

try {
  if (typeof B !== 'undefined' && B.a2aPeers) {
    B.a2aPeers(function(peers) {
      _a2aPeerCache = (peers && peers.length) ? peers : [];
      var box = document.getElementById('a2aPeerList');
      if (!box) return;
      if (peers && peers.length) {
        var h = '';
        peers.forEach(function(p) {
          h += '<div class="peer-item">· ' + escHtml(p.name)
            + ' <span class="peer-id">' + escHtml(p.deviceId) + '</span></div>';
        });
        box.innerHTML = h;
      } else {
        box.innerHTML = '暂无已配对设备';
      }
    });
  }
} catch(e) {}

try {
  if (typeof B !== 'undefined' && B.a2aGetRelay) {
    var url = B.a2aGetRelay();
    var inp = document.getElementById('a2aRelayInput');
    if (inp && url) inp.value = url;
  }
} catch(e) {}
}

/** 保存 relay 地址 (A2aClient 校验 http/https) 并刷新 */
function saveA2aRelay() {
  var inp = document.getElementById('a2aRelayInput');
  var url = (inp && inp.value) ? inp.value.trim() : '';
  try { if (typeof B !== 'undefined' && B.a2aSetRelay) B.a2aSetRelay(url); } catch(e) {}
  redrawSettings();
}

/* — 权限与功能二级页 (工单 14: 待开启组带用途 + 已开启组 + 健康行 + 工具组) — */

function renderPermPage() {
  var out = '';
  try {
    /* 权限分组: 待开启 (未授权, 带用途文案) + 已开启 */
    var perms = (typeof B !== 'undefined' && B.permState) ? B.permState() : {};
    var names = {
      NOTIFY: ['通知', '任务完成提醒'], CAMERA: ['相机', '拍照识别'], LOCATION: ['位置', '出行提醒'],
      CONTACTS: ['通讯录', '联系人协作'], SMS: ['短信', '验证码读取'], CALL: ['电话', '通话服务'],
      SETTINGS: ['写系统设置', '深色模式等系统功能']
    };
  };
  var pending = [], granted = [];
  var keys = Object.keys(perms);

```

```

keys.forEach(function(k) {
  if (!names[k]) return;
  if (perms[k]) granted.push(k); else pending.push(k);
});
if (!keys.length) {
  out += '<div class="dsb meta pad4">权限状态不可用（桥查询失败）</div>';
} else {
  if (pending.length) {
    out += '<div class="set-mini-label">待开启 · 开启后对应功能才可用</div><div class="set-group"
    pending.forEach(function(k) {
      out += '<div class="set-row" onclick="NewFace.openNativePerm()"><div class="t"
        + '<div class="v">' + names[k][1] + '</div><div class="chev">></div></div>';
    });
    out += '</div>';
  }
  if (granted.length) {
    out += '<div class="set-mini-label">已开启</div><div class="set-group">';
    granted.forEach(function(k) {
      out += '<div class="set-row static"><div class="t">' + names[k][0] + '</div>'
        + '<div class="v ok">已开启</div></div>';
    });
    out += '</div>';
  }
  out += '<div class="dact perm-dact" onclick="NewFace.openNativePerm()">打开系统权限设置</d
}
/* H2 健康行 - 照 {id, category, status(ok/warn/down), reason} 渲染每源一行，点按重测 */
try {
  var h = (typeof B !== 'undefined' && B.query) ? B.query({q:'health'}) : null;
  var items = (h && h.ok && h.data && h.data.items) ? h.data.items : [];
  if (items.length) {
    out += '<div class="set-mini-label">功能健康</div><div class="set-group">';
    items.forEach(function(it) {
      var st = it.status || 'ok';
      var color = st === 'ok' ? 'var(--ok)' : (st === 'down' ? '#c2402b' : '#c08552');
      var label = {ok:'正常', warn:'异常', down:'故障'}[st] || st;
      var reason = it.detail || it.reason || '';
      out += '<div class="set-row" onclick="NewFace._healthRetest(\'' + escHtml(it.i
        + '<div class="v" style="color:' + color + '"' + label
        + (st !== 'ok' && reason ? ' · ' + escHtml(reason) : '') + '</div>'
        + (st !== 'ok' ? '<div class="chev">重测</div>' : '') + '</div>';
    });
    out += '</div>';
  }
} catch(e){}
/* 工具组（tool_manifest 分组投影，保持能力区块健康行收此） */
var resp = (typeof B !== 'undefined' && B.query) ? B.query({q:'tool_manifest'}) : nu
var tools = (resp && resp.ok && resp.data && resp.data.tools) ? resp.data.tools : []
var groups = {};
for (var i = 0; i < tools.length; i++) {
  var t = tools[i];

```

```

    var g = t.toolset || 'other';
    if (!groups[g]) groups[g] = [];
    groups[g].push(t);
  }
  var gkeys = Object.keys(groups);
  if (gkeys.length) {
    out += '<div class="set-mini-label">已开启功能（工具）</div><div class="set-group">';
    keys.forEach(function(g) {
      var label = {media:'媒体', travel:'出行', memory:'记忆', web:'网页', system:'系统',
        device:'设备', file:'文件', shell:'命令行', message:'消息'}[g] || g;
      groups[g].forEach(function(t) {
        var ok = t.check == null;
        var badge = ok ? '可用' : (String(t.check).slice(0, 18) || '需授权');
        out += '<div class="set-row static"><div class="t">' + escHtml(t.name) + '</div>'
          + '<div class="v" style="color:' + (ok ? 'var(--ok)' : 'var(--ink-3)') + '>'
          + escHtml(badge) + '</div></div>';
      });
    });
    out += '</div>';
  } else {
    out += '<div class="dsub meta pad6">暂无工具清单（先运行一次慢脑任务，注册表加载后此处自动投影）</div>';
  }
} catch(e) {}
return out;
}
/* H2 件二：健康行点按重测 → 重渲染权限页 */
function _healthRetest(id) {
  try { if (typeof B !== 'undefined' && B.query) B.query({q:'health', op:'retest', id:id
    B.toast('已重测 ' + id);
    setTimeout(function(){ try { redrawSettings(); } catch(e){ } }, 1500);
  }

function openNativePerm() {
  try { if (typeof B !== 'undefined' && B.openAppSettings) B.openAppSettings(); } catch(
}

/* — 保险柜二级页 + 一级「清除对话记录」确认 sheet（工单 14） — */
function renderVaultPage() {
  var out = '';
  /* 状态 + 解锁查看（P5 功能迁入，锁屏闸不变） */
  out += '<div class="set-group">'
    + '<div class="set-row static"><div class="t">状态</div><div class="v">已锁定</div></div>'
    + '<div class="set-row" onclick="NewFace.loadVaultEntries()"><div class="t">解锁查看</di'
    + '</div>';
  /* 条目列表（vaultList 异步投影） */
  out += '<div class="set-group"><div id="vaultEntries"><div class="set-row static"><div'
  /* 清除对话记录（工单 25：从一级页收进信息管理二级页，行为不变仍走 clearRecordsSheet） */
  out += '<div class="set-group">'
    + '<div class="set-row danger" onclick="NewFace.clearRecordsSheet()"><div class="t">

```

```

    + '</div>';
    /* 数据说明（原隐私区块数据说明迁此） */
    out += '<div class="note">保险柜内容经本机硬件密钥加密，解锁需验证锁屏密码。原文永不离开本机。对话 / 追剧 / 观看
记录全部只存在这台手机，不上
    setTimeout(function(){ try { loadVaultEntries(); } catch(e){} }, 30);
    return out;
}

/** 一级「清除对话记录」→ 确认 sheet 内并列选项（施工决策：观看历史/追剧记忆/全部对话记录） */
function clearRecordsSheet() {
    ensureExpertOverlay();
    var overlay = document.getElementById('expertPreviewOverlay'); if (!overlay) return;
    document.getElementById('expertPreviewTitle').textContent = '清除对话记录';
    document.getElementById('expertPreviewMeta').textContent = '以下内容清除后不可恢复';
    var whCount = 0, dmCount = 0;
    try {
        var r1 = (typeof B !== 'undefined' && B.query) ? B.query({q:'watch_history'}) : null;
        if (r1 && r1.ok && r1.data) whCount = Object.keys(r1.data).length;
        var r2 = (typeof B !== 'undefined' && B.query) ? B.query({q:'drama_memory'}) : null;
        if (r2 && r2.ok && r2.data && r2.data.favs) dmCount = r2.data.favs.length;
    } catch(e) {}
    document.getElementById('expertPreviewBody').innerHTML =
        '<div class="confirm-txt">' + esc('观看历史（' + whCount + ' 部记录）') + '</div>'
        + '<div class="abtn" onclick="NewFace._clearRec(\'watch\')">清空观看历史</div>'
        + '<div class="confirm-txt">' + esc('追剧记忆（' + dmCount + ' 部收藏）') + '</div>'
        + '<div class="abtn" onclick="NewFace._clearRec(\'drama\')">清空追剧记忆</div>'
        + '<div class="confirm-txt">' + esc('全部对话记录（所有房间）') + '</div>'
        + '<div class="abtn danger" onclick="NewFace._clearRec(\'all\')">清空全部对话记录</div>';
    document.getElementById('expertPreviewActs').innerHTML =
        '<div class="abtn" onclick="NewFace.expertClosePreview()">取消</div>';
    overlay.style.display = 'flex';
}

function _clearRec(which) {
    try {
        if (which === 'watch') { if (B.query) B.query({q:'watch_history', op:'clear'}); B.to
        else if (which === 'drama') { if (B.query) B.query({q:'drama_memory', op:'clear'});
        else if (which === 'all') {
            /* 全部对话记录：遍历 face_history 房间逐个销毁（room_destroy 删磁盘房） */
            var resp = (typeof B !== 'undefined' && B.query) ? B.query({q:'face_history'}) : n
            var groups = (resp && resp.ok && resp.data && resp.data.groups) ? resp.data.groups
            var n = 0;
            groups.forEach(function(g) {
                (g.items || []).forEach(function(it) { if (it.roomId) { try { B.roomDestroy(it.r
                });
            });
            B.toast(n ? ('已清空 ' + n + ' 个对话') : '没有可清空的对话');
        }
    } catch(e) { B.toast('操作失败'); }
    expertClosePreview();
    redrawSettings();
}

```

```

/* — ⑥ 保险柜（P5 双副本保险柜 · 设置页「隐私」区块条目：列表/解锁查看/删除，屏⑥ 形态） — */
var _vaultEntries = [];
var _vaultPendingDelete = -1;

function loadVaultEntries() {
  var box = document.getElementById('vaultEntries'); if (!box) return;
  if (typeof B === 'undefined' || !B.vaultList) {
    box.innerHTML = '<div class="ditem"><div class="dsub">保险柜桥未就绪</div></div>';
    return;
  }
  B.vaultList(function(res){
    if (!res || !res.ok) {
      var em = (res && res.error && res.error.msg) || '保险柜不可用';
      box.innerHTML = '<div class="ditem"><div class="dsub">' + escHtml(em) + '</div></div>';
      return;
    }
    var entries = (res.data && res.data.entries) || [];
    _vaultEntries = entries;
    if (!entries.length) {
      box.innerHTML = '<div class="ditem"><div class="dsub">空 · 敏感内容会在这里上锁</div></div>';
      return;
    }
    var html = '';
    for (var i = 0; i < entries.length; i++) {
      var e = entries[i];
      html += '<div class="ditem" onclick="NewFace.vaultShowEntry(' + i + ')">🔒 ' + escH
        + '<div class="dsub">' + formatSize(e.sizeBytes) + ' · ' + formatTs(e.createdAtM
      if (_vaultPendingDelete === i) {
        html += '<div class="ditem warn-bold" onclick="NewFace.vaultDeleteEntry(' + i + '
          + '<div class="dsub">不可恢复</div></div>';
      } else {
        html += '<div class="ditem warn" onclick="NewFace.vaultDeleteEntry(' + i + ')">删
          + '<div class="dsub">' + escHtml(e.name) + ' · 点按两次删除</div></div>';
      }
    }
    box.innerHTML = html;
  });
}

/* 解锁查看：过 Keyguard 闸（桥内弹系统指纹/PIN/图案）→ 展示内容 */
function vaultShowEntry(i) {
  var e = _vaultEntries[i]; if (!e) return;
  B.vaultUnlock(e.id, function(res){
    if (!res || !res.ok) {
      var em = (res && res.error && res.error.msg) || '无法解锁';
      B.toast(em);
      return;
    }
  })
  showVaultSheet(e.name, (res.data && res.data.content) || '');
}

```

```

});
}

/* 删除：两次点按确认（不用 confirm(), 守「Sheet 替代弹窗」规则） */
function vaultDeleteEntry(i) {
  if (_vaultPendingDelete !== i) {
    _vaultPendingDelete = i;
    loadVaultEntries();
    return;
  }
  var e = _vaultEntries[i]; if (!e) return;
  B.vaultDelete(e.id, function(res){
    _vaultPendingDelete = -1;
    if (res && res.ok) { B.toast('已从保险柜删除'); }
    else { B.toast((res && res.error && res.error.msg) || '删除失败'); }
    loadVaultEntries();
  });
}

/* 展示解锁内容：独立 fixed sheet（内联样式，textContent 防注入；不走 confirm/prompt） */
function showVaultSheet(name, content) {
  var old = document.getElementById('vaultSheet');
  if (old && old.parentNode) old.parentNode.removeChild(old);
  var sheet = document.createElement('div');
  sheet.id = 'vaultSheet';
  sheet.style.cssText = 'position:fixed;inset:0;background:rgba(28,26,22,.4);display:flex';
  var panel = document.createElement('div');
  panel.style.cssText = 'width:100%;max-height:78%;background:#fefcf9;border-radius:24px';
  var title = document.createElement('div');
  title.textContent = '🔒 ' + (name || '保险柜条目');
  title.style.cssText = 'font-size:17px;font-weight:700;margin-bottom:4px;';
  var meta = document.createElement('div');
  meta.textContent = '保险柜解锁 · 仅本机可见';
  meta.style.cssText = 'font-size:10px;color:#9a938a;margin-bottom:12px;';
  var body = document.createElement('div');
  body.style.cssText = 'flex:1;overflow-y:auto;font-size:12.5px;line-height:2.1;color:#5';
  body.textContent = content;
  var acts = document.createElement('div');
  acts.style.cssText = 'display:flex;gap:10px;margin-top:14px;';
  var close = document.createElement('div');
  close.textContent = '关闭';
  close.style.cssText = 'flex:1;text-align:center;padding:10px 0;border-radius:99px;font';
  close.onclick = function(){ var p = document.getElementById('vaultSheet'); if (p) p.pa
  acts.appendChild(close);
  panel.appendChild(title); panel.appendChild(meta); panel.appendChild(body); panel.appe
  sheet.appendChild(panel);
  document.body.appendChild(sheet);
  sheet.addEventListener('click', function(ev){
    if (ev.target === sheet) { var p = document.getElementById('vaultSheet'); if (p) p.p
  });
}

```

```
}

```

```
/* — ② 技术区块（入口归位，功能仍在原生设置页原样保留） — */

```

```
/* — 派单№5：技术区块 - 外观(主题) + 终端（老运行页入口迁入） — */

```

```
/* — 系统二级页（工单 14：终端/Linux/Hermes/MCP/SSH/诊断/重置 + 工单 15 清理残留房间） — */

```

```
function renderSystemPage() {
    var out = '';
    out += '<div class="set-group">';
    out += '<div class="set-row" onclick="NewFace.openTerminalFromSettings()"><div class="
    + '<div class="v">内嵌 Ubuntu</div><div class="chev"></div></div>';
    out += '<div class="set-row static"><div class="t">Linux 环境</div><div class="v" id="sy
    out += '<div class="set-row static"><div class="t">Hermes 服务</div><div class="v" id="s
    out += '</div>';
    out += '<div class="set-group">';
    out += '<div class="set-row"><div class="t">MCP</div><div class="v">server_url / token
    + '<div class="set-sub"><input id="mcpUrl" placeholder="server_url"><input id="mcpTo
    + '<div class="sysacts"><span class="dact" onclick="NewFace._mcpSave()">保存</span>'
    + '<span class="dact" onclick="NewFace._mcpTest()">测连通</span></div></div>';
    out += '<div class="set-row"><div class="t">SSH（部署服务器）</div><div class="v" id="sysSshS
    + '<div class="set-sub"><div class="sysacts"><span class="dact" onclick="NewFace._ss
    + '<span class="dact" onclick="NewFace._sshTest()">测连接</span></div></div>';
    out += '</div>';
    out += '<div class="set-group">';
    out += '<div class="set-row"><div class="t">导出诊断</div><div class="v">日志 / 状态摘要</div></
    + '<div class="set-sub"><div class="sysacts"><span class="dact" onclick="NewFace._di
    + '<span class="dact" onclick="NewFace._diagClear()">清空</span></div></div>';
    out += '<div class="set-row" onclick="NewFace.cleanupRooms()"><div class="t">清理残留房间</d
    + '<div class="v" id="sysOrphanVal">检查</div><div class="chev"></div></div>';
    out += '</div>';
    out += '<div class="set-group">';
    out += '<div class="set-row danger" onclick="NewFace.resetSystemConfirm()"><div class=
    out += '</div>';
    setTimeout(function(){ try { _renderSysStatus(); } catch(e){} }, 30);
    return out;
}

/* 外观三态（浅色/深色/跟随系统）：mov_theme = light | dark | auto */
function getThemeMode() {
    try { var m = localStorage.getItem('mov_theme'); return (m === 'dark' || m === 'auto')
    }
}
function applyTheme(mode) {
    var dark = (mode === 'dark') || (mode === 'auto' && window.matchMedia && window.matchM
    document.documentElement.classList.toggle('dark', !!dark);
}
function setThemeMode(mode) {
    try { localStorage.setItem('mov_theme', (mode === 'dark' || mode === 'auto') ? mode :
    applyTheme(getThemeMode());
    redrawSettings();
    }
}
```

```

function toggleTheme() { setThemeMode(getThemeMode() === 'dark' ? 'light' : 'dark'); }
function openTerminalFromSettings() {
  closeDrawer();
  try { if (typeof B !== 'undefined' && B.openTerminal) B.openTerminal(); } catch(e){}
}

/* 系统级状态拉取 (Kernel query) + 安装进度轮询 (无感工作态) */
function _sysQuery(q) { try { return B.query(q); } catch(e){ return null; } }
function _fv(id) { var e = document.getElementById(id); return e ? e.value.trim() : ''; }
function _renderSysStatus() {
  var murl = document.getElementById('mcpUrl');
  if (murl && !murl.value) {
    var m = _sysQuery({q:'mcp.get'});
    if (m && m.ok && m.data) {
      murl.value = m.data.server_url || '';
      var mt = document.getElementById('mcpToken'); if (mt) mt.value = m.data.server_tok
    }
  }
  var ssh = (typeof B !== 'undefined' && B.getDeployConfig) ? B.getDeployConfig() : null
  var ss = document.getElementById('sysSshSub');
  if (ss && ssh && ssh.configured) ss.textContent = ssh.host + (ssh.port ? ':' + ssh.port : '');
  /* Linux 环境 / Hermes 服务状态行 */
  try {
    var ls = _sysQuery({q:'linuxStatus'});
    var le = document.getElementById('sysLinuxVal');
    if (le) {
      var okL = ls && ls.ok && ls.data && ls.data.installed;
      le.textContent = okL ? '已安装' : '未安装';
    }
  } catch(e) {}
  try {
    var hs = (typeof B !== 'undefined' && B.hermesStatus) ? B.hermesStatus() : null;
    var he = document.getElementById('sysHermesVal');
    if (he) he.textContent = (hs && hs.ready) ? '运行中' : '未运行';
  } catch(e) {}
}

/** 重置系统: 两步确认 → resetSystem (破坏性操作, 红线确认闹) */
function resetSystemConfirm() {
  expertConfirm(null, '重置系统将清除全部数据并恢复初始状态, 不可恢复. 确定继续?', function(){
    expertConfirm(null, '再次确认: 清除全部数据并恢复初始状态?', function(){
      try { B.postCmd({cmd:'resetSystem'}); B.toast('已重置'); } catch(e){ B.toast('重置失败');
        redrawSettings();
      }
    });
  });
}

/** 清理残留房间 (工单 15: rooms.orphan 查询 → 确认 → rooms.cleanup 执行) */
function cleanupRooms() {
  try {

```

```

    var r = B.postCmd({cmd:'rooms.orphan'});
    var list = (r && r.ok && r.data && r.data.orphans) ? r.data.orphans : [];
    var val = document.getElementById('sysOrphanVal');
    if (val) val.textContent = list.length ? (list.length + ' 个') : '无残留';
    if (!list.length) { B.toast('没有残留房间'); return; }
    expertConfirm(null, '清理 ' + list.length + ' 个残留房间目录? 删除后不可恢复.', function(){
        try { B.postCmd({cmd:'rooms.cleanup'}); B.toast('已清理'); } catch(e){ B.toast('清理失败');
            redrawSettings();
        }
    });
} catch(e) { B.toast('检查失败'); }
}
function _mcpSave() {
    try { B.postCmd({cmd:'mcp.set', server_url:_fv('mcpUrl'), server_token:_fv('mcpToken')
        B.toast('MCP 配置已保存');
    }
}
function _mcpTest() {
    B.toast('测试 MCP 连通...');
    var r = B.postCmd({cmd:'mcp.test', server_url:_fv('mcpUrl'), server_token:_fv('mcpToken')
    B.toast(r && r.ok && r.data && r.data.ok ? 'MCP 连接正常' : 'MCP 连接失败');
}
function _sshEdit() {
    var cfg = (typeof B !== 'undefined' && B.getDeployConfig) ? B.getDeployConfig() : {};
    var overlay = document.getElementById('expertPreviewOverlay'); if (!overlay) return;
    document.getElementById('expertPreviewTitle').textContent = 'SSH 部署服务器';
    document.getElementById('expertPreviewMeta').textContent = '';
    var body = document.getElementById('expertPreviewBody');
    body.innerHTML = '<div class="mform-label">Host</div><input id="sshHost" value="' + es
        + '<div class="mform-label">Port</div><input id="sshPort" value="' + escHtml(cfg.por
        + '<div class="mform-label">User</div><input id="sshUser" value="' + escHtml(cfg.use
        + '<div class="mform-label">Secret</div><input id="sshSecret" value="' + escHtml(cfg
    document.getElementById('expertPreviewActs').innerHTML =
        '<div class="abtn save" onclick="NewFace._sshSave()">保存</div>'
        + '<div class="abtn" onclick="NewFace.expertClosePreview()">取消</div>';
    overlay.style.display = 'flex';
}
function _sshSave() {
    var o = {host:_fv('sshHost'), port:_fv('sshPort'), user:_fv('sshUser'), auth:'key', se
    var r = B.saveDeployConfig(o);
    B.toast(r && r.ok ? 'SSH 已保存' : 'SSH 保存失败');
    expertClosePreview();
    setTimeout(function(){ try { _renderSysStatus(); } catch(e){} }, 500);
}
function _sshTest() {
    B.toast('测试 SSH 连接...');
    if (typeof B.testDeployConnection === 'function') B.testDeployConnection(function(res)
}
function _diagExport() { try { B.postCmd({cmd:'diagnostics.export'}); } catch(e){} B.toa
function _diagClear() { try { B.postCmd({cmd:'diagnostics.clear'}); } catch(e){} B.toast
/** 通用连接入口: 已连接→搜, 未连接→登录 */

```

```

/** 通用连接入口：已连接→搜，未连接→登录 */
function openConnect(platform) {
  closeDrawer();
  var connected = false;
  try { if (typeof B !== 'undefined' && B.connectStatus) connected = !!B.connectStatus(p
  if (connected) {
    var t = el('<div class="ctxline"><b>◆</b><span>' + (platform === 'douyin' ? '抖音' : '
    document.getElementById('faceCover').appendChild(t);
    setTimeout(function(){ if (t.parentNode) t.parentNode.removeChild(t); }, 2400);
    return;
  }
  if (typeof B !== 'undefined' && B.openConnectLogin) B.openConnectLogin(platform);
}

/** 视频源注册表区块：只显示可用源；blocked/已关的不出现(注册表 JSON 留档) */
function renderSourcesSection() {
  var out = '<div class="dgroup">视频源</div>';
  try {
    var resp = B.query({q:'video_sources'});
    var list = (resp && resp.ok && resp.sources) ? resp.sources : [];
    var DOT = {good:'#5d9b4a', session:'#4a6fd4', limited:'#c8a13a', blocked:'#b8b2a8'};
    var hidden = 0;
    list.forEach(function(s) {
      if (s.status === 'blocked' || !s.enabled) { hidden++; return; } // 下架/已关不显示
      out += '<div class="ditem src-row" onclick="NewFace.toggleSource(\'' + escHtml(s.i
      + '<span class="src-dot" style="background:' + (DOT[s.status] || '#b8b2a8') + '"
      + '<span class="f-grow"><span class="src-name">' + escHtml(s.name) + '</span>'
      + '<span class="meta ml6">' + escHtml(s.scope) + '</span>'
      + '<div class="meta mt2">' + escHtml(s.note) + '</div></span>'
      + '<span class="src-on">启用</span></div>';
    });
    if (hidden > 0) {
      out += '<div class="meta pad6">' + hidden + ' 个源已下架（注册表留档，不再出现）</div>';
    }
  } catch(e) {}
  return out;
}

/** 切换视频源开关并刷新设置页 */
function toggleSource(id, enabled) {
  try { B.query({q:'video_sources', action:'toggle', id:id, enabled:enabled}); } catch(e
  redrawSettings();
}

function openDouyin(query) {
  closeDrawer();
  var connected = false;
  try { if (typeof B !== 'undefined' && B.connectStatus) connected = !!B.connectStatus('

```

```

    if (connected) {
        if (query) { if (typeof B !== 'undefined' && B.openConnectSearch) B.openConnectSearch
            var t = el('<div class="ctxline"><b>◆</b><span>抖音已连接 · 直接说「我要追 xx」就能看</span></div>');
            document.getElementById('faceCover').appendChild(t);
            setTimeout(function(){ if (t.parentNode) t.parentNode.removeChild(t); }, 2400);
            return;
        }
        if (typeof B !== 'undefined' && B.openConnectLogin) B.openConnectLogin('douyin');
    }
}

function previewMine(roomId, fileName) {
    closeDrawer();
    /* 第二幕：文件预览统一走新脸 overlay 体系 (home 与专家共用 expertPreviewOverlay)，不再 faceCover 全屏 */
    expertPreviewFile(roomId, fileName);
}

function closeDrawer() {
    var mask = document.getElementById('faceDmask');
    var drawer = document.getElementById('faceDrawer');
    if (mask) mask.classList.remove('on');
    if (drawer) drawer.classList.remove('on');
}

function resetFace() {
    // 清理所有 overlay
    closePaySheet(); closePlayer(); closeEmbedPlayer();
    var dh = document.getElementById('faceDoneHost'); if (dh) dh.innerHTML = '';
    var ph = document.getElementById('facePlayerHost'); if (ph) ph.innerHTML = '';
    // 清理 A2A 状态
    if (_a2a && _a2a.pollTimer) { clearInterval(_a2a.pollTimer); _a2a.pollTimer = null; }
    try { if (typeof B !== 'undefined' && B.a2aStopPolling) B.a2aStopPolling(); } catch(e)
    _a2a = { state: 'idle', peer: null, cart: [], confirm: null, orderId: null, pollTimer:
    // 恢复封面
    var cover = document.getElementById('faceCover');
    if (cover && _heroHTML) cover.innerHTML = _heroHTML;
    _chatStreamOn = false; // 工单 17: 恢复封面 = 新会话，下次 say 重新初始化对话流
    _homeRoomId = null; // 工单 17: 回封面 = 新会话，下次派活开新房
    var hero = document.getElementById('faceHero');
    if (hero) hero.style.display = '';
    // 2C.5: 恢复封面后重新注入"接着上次看" (_heroHTML 快照不含注入卡)
    injectContinueWatching();
    // FUSION F5: 恢复封面后若有最近慢脑交付 → 回脸交付卡
    renderFaceDelivery();
}

function submitIntent() {
    var input = document.getElementById('faceInput');
    if (!input) return;

```

```

var v = input.value.trim();
if (!v) return;
input.value = '';
say(v);
}

function bindMic() {
var mic = document.getElementById('faceMic');
var pill = document.getElementById('facePill');
var input = document.getElementById('faceInput');
if (!mic || !pill || !input) return;
/* W4: 真语音 - 按住 → B.startSTT(系统 RecognizerIntent) → HermesActivity 9002 回 _movStt → in
window._movStt = function(res) {
pill.classList.remove('listening'); input.classList.remove('listen-hint');
if (res && res.ok && res.text) {
input.placeholder = '说一句话...';
input.value = res.text;
submitIntent();
} else {
input.placeholder = (res && res.error === 'speech_cancelled') ? '说一句话...' : '语音不可用 .
};
var listening = false;
mic.addEventListener('pointerdown', function(e) {
e.preventDefault(); listening = true;
pill.classList.add('listening');
input.value = ''; input.placeholder = '正在听...松开结束';
input.classList.add('listen-hint');
try { if (typeof B !== 'undefined' && B.startSTT) B.startSTT(); } catch(err) {}
});
mic.addEventListener('pointerup', function() { listening = false; });
mic.addEventListener('pointerleave', function() {
if (listening) { listening = false; pill.classList.remove('listening'); input.classL
});
}

function escHtml(s) { return String(s).replace(/&/g, '&amp;').replace(/</g, '&lt;').replac

/* — 初始化 — */
var _heroHTML = '';

function init() {
var el = document.getElementById('view-face');
if (!el) return;
// 快照封面 HTML (say() 会替换, resetFace() 恢复用)
var cover = document.getElementById('faceCover');
if (cover) _heroHTML = cover.innerHTML;
// 日期行
var dl = document.getElementById('faceDateline');
if (dl) { var d = new Date(), wk = '日一二三四五六'[d.getDay()]; dl.textContent = (d.getMonth

```

```

// 默认显示新脸 (debug 包)
if (typeof curTab !== 'undefined') curTab = 'face';
showFace();
// 2C.5: 注入"接着上次看"情境卡
setTimeout(function() { injectContinueWatching(); scanDramaUpdates(); }, 300);
// 更新雷达每 6h 重扫
setInterval(function() { scanDramaUpdates(); }, 2160000);
// 绑定输入 Enter
var input = document.getElementById('faceInput');
if (input) input.addEventListener('keydown', function(e) { if (e.key === 'Enter') subm
bindMic();
}

document.addEventListener('DOMContentLoaded', init);
// FUSION F5: 加载即注册慢脑 deliver 事件监听 (bridge.js 先于本文件加载, _movOn 已就绪)
initMovListener();
return {
  showFace: showFace, switchToExpert: switchToExpert,
  /* mov-verify 兼容: 暴露真实进房路径 (expertOpenRoom 原为新脸 IIFE 局部, mov-verify room 命令复用) */
  openRoom: expertOpenRoom,
  say: say, voice: voice,
  openDrawer: openDrawer, closeDrawer: closeDrawer,
  resetFace: resetFace, openSettings: openSettings,
  closeSettings: closeSettings, settingsGo: settingsGo, settingsBack: settingsBack,
  redrawSettings: redrawSettings, checkUpdate: checkUpdate,
  merchantSave: merchantSave,
  setThemeMode: setThemeMode, setRoutingMode: setRoutingMode,
  clearRecordsSheet: clearRecordsSheet, _clearRec: _clearRec,
  _fbToggleDiag: _fbToggleDiag, _fbSubmit: _fbSubmit,
  resetSystemConfirm: resetSystemConfirm, cleanupRooms: cleanupRooms,
  clearConnect: clearConnect,
  openNativePerm: openNativePerm, gotoDelivery: gotoDelivery,
  genA2aPairCode: genA2aPairCode, saveA2aRelay: saveA2aRelay,
  deleteLocalModel: deleteLocalModel,
  submitIntent: submitIntent, showEmpty: showEmpty,
  cycleSlot: cycleSlot, learnAnswer: learnAnswer,
  failRetry: failRetry, failAlt: failAlt,
  closePlayer: closePlayer, startFail: startFail,
  handoffToExpert: handoffToExpert,
  _a2aSelectPeer: _a2aSelectPeer, _a2aPair: _a2aPair,
  _a2aCartChange: _a2aCartChange, _a2aSubmitOrder: _a2aSubmitOrder,
  _a2aIPaid: _a2aIPaid, _a2aOpenPayLink: _a2aOpenPayLink,
  _a2aCopyNote: _a2aCopyNote, _a2aWechatScan: _a2aWechatScan,
  _a2aSubmitReview: _a2aSubmitReview, // W1 第二幕: 随机抽评提交
  _continueSearch: _continueSearch,
  _musicToggle: _musicToggle, _musicStop: _musicStop, _musicSeek: _musicSeek,
  _dramaFav: _dramaFav, _dramaBlock: _dramaBlock,
  _openConnectUrl: _openConnectUrl,
  openDouyin: openDouyin,
  openConnect: openConnect,

```

```

toggleSource: toggleSource,
replayConversation: replayConversation, openRoomFromHistory: openRoomFromHistory, prev
expertOpenRoom: expertOpenRoom, expertBack: expertBack,
expertSend: expertSend, expertNewRoom: expertNewRoom, expertExit: expertExit,
expertGoRun: expertGoRun, expertGoFiles: expertGoFiles, expertBackToRooms: expertBackT
expertStopTask: expertStopTask,
expertPreviewFile: expertPreviewFile, expertClosePreview: expertClosePreview,
expertEditPreview: expertEditPreview, expertSavePreview: expertSavePreview,
expertExportPreview: expertExportPreview, expertContinuePreview: expertContinuePreview
expertInstallPreview: expertInstallPreview, expertVaultOpen: expertVaultOpen,
loadVaultEntries: loadVaultEntries, vaultShowEntry: vaultShowEntry, vaultDeleteEntry:
expertRoomOps: expertRoomOps, expertOpsClose: expertOpsClose,
expertOpsRename: expertOpsRename, expertOpsRenameOk: expertOpsRenameOk,
expertOpsArchive: expertOpsArchive, expertOpsClear: expertOpsClear, expertOpsDelete: e
_expertConfirmOk: _expertConfirmOk,
expertVersionsPreview: expertVersionsPreview, expertBackFromVersions: expertBackFromVe
expertRestoreVersion: expertRestoreVersion,
_mops: _mops, _mopsClose: _mopsClose, _msetDefault: _msetDefault, _mtest: _mtest,
_medit: _medit, _mdelete: _mdelete, _mform: _mform, _mfProvider: _mfProvider,
_mformCancel: _mformCancel, _msave: _msave, _mtestForm: _mtestForm,
toggleTheme: toggleTheme, openTerminalFromSettings: openTerminalFromSettings,
_mcpSave: _mcpSave, _mcpTest: _mcpTest,
_sshEdit: _sshEdit, _sshSave: _sshSave, _sshTest: _sshTest,
_diagExport: _diagExport, _diagClear: _diagClear, _healthRetest: _healthRetest,
startShopSearch: startShopSearch
});
})();
app.js
/* =====
app.js - 入口: 初始化 + 全局事件
第三幕清场: btnBack 老视图绑定已删 (专家模式新脸化接管房间/会话/文件/运行);
app-chat/app-files 纯老视图 JS 已删; 新脸 init 由 newface.js showFace() 接管
===== */

/* ===== 初始化 ===== */
initLang();
applyI18n();
if(typeof refreshModelAvatars==='function')refreshModelAvatars();
/* P2: 投影渲染器由 enterRoom 参数化 init 到注入容器 (chatBody 老 DOM 已删, 不再全局 init);
refreshRuntime 已随老运行页删除 (派单№5: 设置迁新脸抽屉, TOKEN/模型由抽屉区块渲染) */
ev('MOV v3.0 '+t('ready')+(B.present?' · '+t('bridge.on'):' · '+t('bridge.off')));

/* 加密状态检查 */
if(B.present){
  var enc=B.encStatus();
  if(enc&&!enc.ok){
    setTimeout(function(){B.toast('△ 加密存储不可用, API Key 以明文存储');}, 2000);
  }
}
}
mov-gig.js

```

```

/* — MOV · 我的能力（职业体系 G1a 前端，G0 relay 已就绪）
入口：设置 > 账户管理 > 我的能力（renderAbility 三级页）
桥契约（BridgeFactory，已就绪）：
    HermesBridge.a2aRegisterPro(deviceId, name, role, professionsJson, cbId)
    — 上报 role=pro + professions（职业注册表，relay 侧 G0 已扩展）
    HermesBridge.a2aRegistered() / a2aDeviceId()
    HermesBridge.a2aQueryPro(craft, radiusKm, online, cbId)（G1b 用）
状态：localStorage 'mv:gig:profs'（非敏感：craft/form/radiusKm/online 列表）
    localStorage 'mv:gig:declared'（本地声明降级，G1 认证闸降级口径）
工种 11 枚举（DESIGN_GIG_REGISTRY §二分类表）：form 按工种自动推导，半径默认 2km
buyer 无新入口：消费走对话 gig.*（G5）—本页只管 pro 自我管理。*/
window.MovGig = (function() {

    /* == 工种枚举（11 项，对齐 relay CRAFTS + DESIGN 分类表） == */

    var CRAFTS = [
        { craft:'shop',      name:'卖货',    icon:'🏪', form:'instant',    cat:'卖货型' },
        { craft:'food',      name:'餐饮',    icon:'🍴', form:'instant',    cat:'卖货型' },
        { craft:'delivery',  name:'外卖跑腿', icon:'🚶', form:'instant',    cat:'运力型' },
        { craft:'express',   name:'快递',    icon:'📦', form:'instant',    cat:'运力型' },
        { craft:'taxi',      name:'出租车',  icon:'🚗', form:'instant',    cat:'运力型' },
        { craft:'barber',    name:'理发师',  icon:'💇', form:'appointment', cat:'手艺·到店' },
        { craft:'nail',      name:'美甲师',  icon:'💅', form:'appointment', cat:'手艺·到店' },
        { craft:'massage',   name:'按摩',    icon:'💆', form:'appointment', cat:'手艺·到店' },
        { craft:'tutor',     name:'家教',    icon:'👨🏫', form:'appointment', cat:'手艺·上门' },
        { craft:'repair',    name:'维修',    icon:'🔧', form:'appointment', cat:'手艺·上门' },
        { craft:'cleaning',  name:'保洁',    icon:'🧹', form:'appointment', cat:'手艺·上门' }
    ];

    /* == 状态（非敏感，localStorage 可存） == */

    var KEY_PROFS = 'mv:gig:profs';
    var KEY_DECLARED = 'mv:gig:declared';
    var KEY_PENDING = 'mv:gig:pending'; // 开通页未提交的多选草稿

    var profs = []; // [{craft, form, radiusKm, online}]
    var declared = false; // 本地声明（个人认证降级）
    var pending = []; // 开通页多选草稿
    var adding = false; // 添加工种模式（已开通者重进开通页）
    var registered = false;

    function load() {
        try {
            profs = JSON.parse(localStorage.getItem(KEY_PROFS) || '[]');
            declared = localStorage.getItem(KEY_DECLARED) === '1';
            pending = JSON.parse(localStorage.getItem(KEY_PENDING) || '[]');
        } catch(e) { profs = []; declared = false; pending = []; }
        try { registered = !!window.HermesBridge && HermesBridge.a2aRegistered && HermesBridg
        }
    }
    function saveProfs() {

```

```

    try { localStorage.setItem(KEY_PROFS, JSON.stringify(profs)); } catch(e) {}
  }
  function savePending() {
    try { localStorage.setItem(KEY_PENDING, JSON.stringify(pending)); } catch(e) {}
  }

  /* == 工具 == */

  function esc(s) {
    return String(s == null ? '' : s).replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/
    .replace(/"/g, '&quot;').replace(/'/g, '&#39;');
  }
  function toast(m) { try { if (typeof B !== 'undefined' && B.toast) B.toast(m); } catch(e) {} }
  function refresh() { try { NewFace.redrawSettings(); } catch(e) {} }
  function craftInfo(craft) {
    for (var i = 0; i < CRAFTS.length; i++) if (CRAFTS[i].craft === craft) return CRAFTS[i]
    return null;
  }

  /* == 摘要（账户管理行右侧） == */

  function summary() {
    if (!profs.length) return '未开通';
    var on = 0;
    profs.forEach(function(p) { if (p.online) on++; });
    var names = profs.slice(0, 2).map(function(p) {
      var c = craftInfo(p.craft);
      return c ? c.name : p.craft;
    }).join('·');
    if (profs.length > 2) names += '...';
    return names + (on > 0 ? '·接单中' : '·休息中');
  }

  /* == 渲染：我的能力三级页 == */

  function renderAbility() {
    var html = '<div class="set-topbar"><span class="set-back" onclick="NewFace.settingsBa
    + '<span class="set-ttl">我的能力</span></div>';

    if (!profs.length || adding) {
      html += renderOpenPage();      // 未开通 → 开通页（只选工种）；添加工种 → 同页预选
    } else {
      html += renderListPage();      // 已开通 → 工种列表 + 添加工种 + 日对账占位
    }
    return html;
  }

  /* 开通页：11 工种宫格多选 + 一键开通（服务形态自动推导，半径默认 2km） */
  function renderOpenPage() {
    var html = '<div class="set-sub" style="padding-top:12px">选择你的职业能力（可多选）</div>'

```

```

+ '<div class="set-group">';
html += '<div class="mk-hint" style="padding:0 12px 8px;font-size:11px;color:var(--ink'
html += '<div style="display:grid;grid-template-columns:repeat(2,1fr);gap:8px;padding:
CRAFTS.forEach(function(c) {
  var on = pending.indexOf(c.craft) >= 0;
  html += '<div class="gig-craft" + (on ? ' on' : '') + '" style="border:1px solid ' +
  + ' onclick="MovGig.togglePending(\'\' + c.craft + '\')">'
  + '<div style="font-size:22px">' + c.icon + '</div>'
  + '<div style="font-weight:600;margin-top:4px">' + c.name + '</div>'
  + '<div style="font-size:11px;color:var(--ink-3,#888)">' + c.cat + ' · ' + (c.form
  + (on ? '<div style="font-size:12px;color:var(--accent,#3b82f6);margin-top:4px">✓
  + '</div>';
});
html += '</div></div>';

/* 前置闸：个人认证（v1 本地声明降级） */
var gate = declared;
html += '<div class="set-group">'
+ '<div class="set-row static"><div class="t">个人认证</div><div class="v">' + (declared
+ (declared
  ? '<div class="note">已本地声明，开通后职业能力即时生效（短信实名通道接入后自动升级）。</div>'
  : '<div class="note">未认证：v1 以本地声明降级（认证徽章不点亮）。</div>')
+ '</div>';

html += '<div class="mk-buybar" style="margin-top:10px;display:flex;gap:8px">'
+ '<span class="bb ghost" style="flex:1;text-align:center;padding:10px 0;border-radi
+ '<span id="gigOpenBtn" class="bb solid" style="flex:2;text-align:center;padding:10
+ '</div>';
if (!gate) html += '<div class="note">请先完成「本地声明」解锁开通。</div>';
return html;
}

/* 已开通：工种列表（行=图标+名+开关）+ 添加工种 + 日对账占位 */
function renderListPage() {
  var html = '<div class="set-group">';
  profs.forEach(function(p) {
    var c = craftInfo(p.craft) || { name:p.craft, icon:'🔧' };
    html += '<div class="set-row static"><div class="t">' + c.icon + ' ' + c.name
    + '<div class="sub">' + (c.form === 'instant' ? '即时单' : '预约单') + ' · 半径 ' + (p.rad
    + '<div class="v"><span class="switch" + (p.online ? ' on' : '') + '" onclick="Mov
  });
  html += '</div>'
  + '<div class="mk-buybar" style="margin-top:10px">'
  + '<span class="dact" onclick="MovGig.addCraft()">+ 添加工种</span></div>'
  + '<div class="set-group" style="margin-top:10px">'
  + '<div class="set-row static"><div class="t">日对账</div><div class="v">G4 落地后开通</div>
  + '</div>'
  + '<div class="note">开关即接单状态（不设总开关）：任一工种开 = 接单中；状态实时上报职业注册表。</div>';
  return html;
}

```

```
/* == 动作 == */
```

```
function togglePending(craft) {  
  var i = pending.indexOf(craft);  
  if (i >= 0) pending.splice(i, 1); else pending.push(craft);  
  savePending();  
  refresh();  
}
```

```
function declare() {  
  try { localStorage.setItem(KEY_DECLARED, '1'); declared = true; } catch(e) {}  
  toast('已本地声明');  
  refresh();  
}
```

```
/** 一键开通: a2aRegisterPro 上报 role=pro + professions → 本地落盘 */
```

```
function open() {  
  if (!declared) { toast('请先完成本地声明'); return; }  
  if (!pending.length) { toast('请至少选择一个工种'); return; }  
  if (!window.HermesBridge || !window.HermesBridge.a2aRegisterPro) { toast('注册桥不可用'); re  
  var list = [];  
  pending.forEach(function(craft) {  
    var c = craftInfo(craft);  
    list.push({ craft: craft, form: c ? c.form : 'instant', radiusKm: 2, online: true })  
  });  
  var btn = document.getElementById('gigOpenBtn');  
  if (btn) btn.textContent = '开通中...';  
  var id = (typeof nextCbId === 'function') ? nextCbId() : ('cb' + Date.now());  
  if (typeof _cbMap !== 'undefined') _cbMap[id] = function(r) {  
    if (btn) btn.textContent = '开通';  
    if (r && r.ok) {  
      profs = list;  
      saveProfs();  
      pending = [];  
      savePending();  
      adding = false;  
      toast('已开通, 开始接单');  
      refresh();  
    } else {  
      toast((r && r.error && r.error.msg) ? ('开通失败: ' + r.error.msg) : '开通失败');  
    }  
  }  
  };  
  var deviceId = '';  
  try { deviceId = HermesBridge.a2aDeviceId ? HermesBridge.a2aDeviceId() : ''; } catch(e)  
  HermesBridge.a2aRegisterPro(deviceId || 'mov-pro', '本地职业人', 'pro', JSON.stringify(list  
}
```

```
/** 工种开关 (接单状态) → 更新本地 + 重上报 */
```

```
function toggleOnline(craft) {
```

```

var found = null;
profs.forEach(function(p) { if (p.craft === craft) found = p; });
if (!found) return;
found.online = !found.online;
saveProfs();
if (window.HermesBridge && HermesBridge.a2aRegisterPro) {
  var id = (typeof nextCbId === 'function') ? nextCbId() : ('cb' + Date.now());
  if (typeof _cbMap !== 'undefined') _cbMap[id] = function(r) {
    if (r && r.ok) toast(found.online ? '已开始接单' : '已暂停接单');
    else { found.online = !found.online; saveProfs(); toast('状态上报失败, 已回滚'); }
    refresh();
  };
  var deviceId = '';
  try { deviceId = HermesBridge.a2aDeviceId ? HermesBridge.a2aDeviceId() : ''; } catch
  HermesBridge.a2aRegisterPro(deviceId || 'mov-pro', '本地职业人', 'pro', JSON.stringify(pr
  )
  refresh();
}

/** 添加工种: 进开通页 (预选已开通的, 新增可叠加; 开通按钮全量上报) */
function addCraft() {
  adding = true;
  pending = [];
  profs.forEach(function(p) { pending.push(p.craft); });
  savePending();
  refresh();
}

/* == after: 载入状态 == */

function after() {
  load();
  /* 已开通但 relay 侧可能未同步 (重启后) → 补一次上报 (幂等) */
  if (profs.length && window.HermesBridge && HermesBridge.a2aRegisterPro) {
    var id = (typeof nextCbId === 'function') ? nextCbId() : ('cb' + Date.now());
    if (typeof _cbMap !== 'undefined') _cbMap[id] = function() {};
    var deviceId = '';
    try { deviceId = HermesBridge.a2aDeviceId ? HermesBridge.a2aDeviceId() : ''; } catch
    HermesBridge.a2aRegisterPro(deviceId || 'mov-pro', '本地职业人', 'pro', JSON.stringify(pr
    )
  }
}

/* == 导出 == */

return {
  renderAbility: renderAbility, after: after, summary: summary,
  togglePending: togglePending, declare: declare, open: open,
  toggleOnline: toggleOnline, addCraft: addCraft
};
})();

```